Outline

AXION Test Suite UI Overhaul Outline

1. Overhaul Goal

Turn the current `App/` test suite into a **premium lab operating system** instead of a plain dev dashboard.

Target feel:

- cinematic
- high-control
- modular
- powerful
- live
- clean, not cluttered
- “Tony Stark lab console,” but still practical for daily work

Core outcome:

- not just prettier
- fully interactive
- faster workflows
- better visibility into runs, gates, artifacts, failures, and system state
- designed to be the main environment you actually want to sit in for hours

---

2. Design Direction

2.1 Visual Identity

A dark, high-tech interface with:

- deep charcoal / obsidian base
- glass panels
- soft glow accents
- precise borders
- subtle gradients
- premium typography hierarchy
- live status colors that feel intentional, not default Tailwind

Mood:

- command center
- R&D lab
- mission control
- premium enterprise intelligence system

2.2 Style Principles

- dark mode first
- layered panels instead of flat pages
- strong spacing rhythm
- dense information, but visually organized
- motion used for meaning, not decoration
- every major action should feel like operating a system, not clicking a form

---

3. UX Philosophy

The new UI should be built around 4 feelings:

A. Control

You should always know:

- what is running
- what failed
- what is blocked
- what needs action
- what changed

B. Speed

Common actions should take minimal clicks:

- start run
- restart stage
- inspect gate failure
- open artifact
- compare outputs
- export bundle

C. Situational Awareness

The system should surface:

- active runs

- recent failures
- health anomalies
- artifact generation state
- maintenance signals
- verification status

D. Depth

The interface should support shallow and deep use:

- quick glance for status
- deep inspection for debugging and system analysis

---

4. New Product Framing

Instead of “pages,” the suite should feel like **modules inside one operating environment**.

Suggested mental model:

- **Command Center** – overall status
- **Runs** – all assemblies and execution activity
- **Workbench** – active assembly deep view
- **Artifacts** – files, outputs, previews
- **Knowledge** – libraries and references
- **Health** – system telemetry and logs
- **Maintenance** – upgrade, repair, verification modes
- **Export** – package and release outputs

This makes the app feel like a real system, not a collection of utility screens.

---

5. Proposed App Information Architecture

5.1 Primary Navigation

Rework the sidebar into grouped navigation.

Group 1 — Core Ops

- Command Center
- New Run
- Runs
- Workbench
- Artifacts

Group 2 — Intelligence

- Features
- Libraries
- Doc Inventory
- Verification

Group 3 — System State

- Health
- Logs
- Telemetry

- Audit
- Maintenance

Group 4 — Output

- Exports
- Packages
- Snapshots

This is cleaner than a long flat sidebar.

5.2 Secondary Context Navigation

Inside major modules, use local tabs or subnav.

Example for Workbench:

- Overview
- Pipeline
- Intake
- Gates
- Artifacts
- Diff
- Config
- Activity

---

6. Main Screens To Redesign

6.1 Command Center

This becomes the hero screen.

Purpose:

- immediate system awareness
- launch point for daily work

Sections:

- hero command bar
- active runs strip
- system health snapshot
- recent failures
- gate alerts
- latest completed kits
- quick actions
- activity timeline

Visual feel:

- large status cards
- animated live indicators
- central “mission control” layout

6.2 Runs Page

A serious run management interface, not just a list.

Should support:

- search
- filters
- status chips
- sort by latest / failed / longest / completed
- bulk actions
- quick open
- rerun / cancel / delete
- compare runs

Layout:

- left: filters and saved views
- center: runs table/cards
- right: quick detail inspector

6.3 Workbench / Assembly Detail

This should become the most powerful screen in the app.

Purpose:

- one place to operate a single run completely

Should include:

- run header with status, timing, IDs, quick actions
- pipeline map
- stage detail panel
- gate results

- intake preview
- artifact explorer
- proof / verification inspector
- config snapshot
- activity log

This is where the “Tony Stark” feeling matters most.

6.4 Artifact Explorer

Not just file browsing.

Should support:

- tree navigation
- preview pane
- JSON/code pretty view
- diff mode
- metadata panel
- download actions
- favorite/pin important artifacts
- recent artifacts

6.5 Maintenance Center

This should feel like a controlled systems engineering panel.

Possible modules:

- health check

- verification run
- coverage audit
- drift detection
- dependency refresh
- rollback / snapshot
- emergency recovery

Each mode should have:

- purpose
- risk level
- required inputs
- run action
- expected outputs
- history

---

7. Key Workflow Improvements

The UI overhaul should improve actual operation, not just aesthetics.

7.1 Workflow: Start New Run

Future flow:

- guided intake
- progress memory
- section completeness

- live validation
- scope summary before launch
- launch into active workbench immediately

7.2 Workflow: Inspect Failure

Future flow:

- click failed run
- jump directly to failing stage
- see gate rule, evidence, outputs, logs
- suggested next actions panel
- rerun stage from same screen

7.3 Workflow: Review Artifacts

Future flow:

- open artifact explorer
- preview rendered docs and JSON
- compare artifact versions between runs
- export specific artifact or whole kit

7.4 Workflow: System Monitoring

Future flow:

- health anomalies surface automatically
- logs and telemetry tied to specific runs

- maintenance actions launch from detected issue context

---

8. Core UI Components To Introduce

8.1 Shell Components

- command sidebar
- top command bar
- breadcrumbs
- workspace header
- global search / quick action palette

8.2 Data Display Components

- premium stat cards
- run status matrix
- pipeline stage rail
- gate result cards
- artifact tree
- structured inspector panels
- activity timeline
- diff viewer
- telemetry mini charts

8.3 Action Components

- launch run modal
- rerun stage modal
- maintenance mode launcher
- export drawer
- command palette
- quick actions dock

8.4 Feedback Components

- live status beacon
- notification tray
- inline system warnings
- progress overlays
- empty states that still feel premium

---

9. Visual Systems

9.1 Layout System

Use a layered app shell:

- sidebar
- topbar
- content canvas
- right-side inspector where useful

9.2 Card System

Cards should vary by purpose:

- hero cards
- system cards
- inspector cards
- warning cards
- execution cards

9.3 Motion System

Use motion lightly for:

- page transitions
- drawer openings
- stage progress
- run status changes
- hover response
- command confirmation

9.4 Color Logic

Color should communicate state:

- neutral = inactive/info
- blue/cyan = processing
- green = success

- amber = warning
- red = failure
- purple = special/system intelligence surfaces

---

10. Functional Standards For The Redesign

This overhaul should preserve full usability.

Non-negotiables:

- all current pages remain functional or get upgraded equivalents
- no fake dashboards
- every critical object should be actionable
- no dead-end screens
- no read-only pages where workflow should exist
- tables and inspectors must support real use
- status and alerts must reflect live backend data
- assembly/workbench must support full run operation

---

11. Build Phases

Phase 1 — Foundation

- app shell redesign
- sidebar regrouping

- top command bar
- visual system
- reusable cards/panels/buttons/tables

Phase 2 — Command Center

- dashboard rebuilt into real control center
- active run visibility
- alerts
- health summary
- quick actions

Phase 3 — Runs + Workbench

- assemblies list overhaul
- assembly detail becomes full workbench
- stage actions
- gate inspection
- artifact integration

Phase 4 — Artifact + Library Systems

- artifact explorer
- better doc/library browsing
- preview modes

- filtering and compare tools

Phase 5 — Maintenance + Telemetry

- maintenance control center
- telemetry visualization
- log correlation
- system activity history

Phase 6 — Polish

- motion
- command palette
- saved views
- keyboard shortcuts
- density tuning
- final visual refinement

---

12. Final Target State

When complete, the UI should feel like:

- a premium internal operating platform
- a futuristic but practical engineering lab console
- a place to actively run, inspect, debug, verify, and package work
- something that is impressive visually but even more impressive operationally

The end result should not feel like:

- a CRUD admin panel
- a dev-only internal tool
- a static documentation surface
- a collection of disconnected pages

It should feel like:

- **AXION Mission Control**

Screens

AXION Test Suite UI Overhaul Blueprint

1. New top-level product shape

Rename the experience mentally from a dev dashboard to a **lab operating system**.

New shell structure

- **Command Center**
- **Runs**
- **Workbench**
- **Artifacts**
- **Knowledge**
- **System**
- **Export**

This maps cleanly onto what you already have, but with a much better user experience.

---

2. New sidebar structure

Your current sidebar is too flat and too library-heavy up front. It needs hierarchy.

2.1 Sidebar groups

Core Ops

- Command Center
- New Run

- Runs
- Workbench
- Artifacts

Intelligence

- Features
- Knowledge Library
- Doc Inventory
- Verification

System

- Health
- Logs
- Maintenance
- Audit
- Telemetry

Output

- Export
- Packages
- Snapshots

2.2 Sidebar behavior

The sidebar should:

- support collapsed and expanded mode
- show section grouping
- show live counts beside important items
- highlight active run state
- surface alerts in navigation, not just inside pages

Example:

- Runs `(3 active)`
- Health `(1 warning)`
- Logs `(12 new)`
- Export `(2 ready)`

2.3 Sidebar feel

Instead of a plain list:

- grouped panels
- soft glass background
- thin gridline borders
- glow state for selected item
- tiny status pips beside critical modules

---

3. App shell layout

The new app shell should have 4 persistent structural zones.

3.1 Left rail

Navigation and system identity.

3.2 Top command bar

Contains:

- global search
- command palette trigger
- quick actions
- environment selector
- active run chip
- notifications tray
- user/settings area

3.3 Main canvas

The current working page.

3.4 Optional right inspector

Used for:

- selected run summary
- artifact metadata
- gate failure details
- health warnings
- quick preview panes

This is important because it makes the UI feel like a real workstation.

---

4. Screen-by-screen blueprint

4.1 Command Center

This replaces the current plain dashboard feel.

Purpose

One-glance system awareness and fast entry into work.

Layout

Top hero strip

- title: `AXION Mission Control`
- live status chip
- active environment
- current stage/gate system summary
- quick launch actions

Row 1: Executive status cards

- Active Runs
- Failed Runs
- Completed Today
- Gates Blocked
- Artifacts Ready

- System Health

These should be premium metric cards, not plain boxes.

Row 2: Live operations area

Split layout:

- left: active run cards
- center: system activity timeline
- right: alerts / blockers / operator actions

Row 3: Command modules

- Start New Run
- Resume Run
- Open Latest Workbench
- Review Failures
- Open Artifact Explorer
- Run Maintenance Scan

Row 4: Recent output strip

- latest kits
- latest exports
- latest proof reports
- latest changed docs

Feeling

This page should feel alive, not static.

4.2 Runs page

This is the upgraded assemblies list.

Purpose

Run fleet management.

Layout

Left filter rail

- search
- status
- date range
- stage
- gate result
- project type
- environment
- saved views

Center main panel

Runs shown in switchable modes:

- table view
- cards view

- timeline view

Right inspector

When a run is selected:

- run ID
- name
- status
- start/end time
- duration
- current stage
- failed gate
- quick actions

Actions

- open workbench
- rerun
- resume
- cancel
- clone run
- export run
- delete
- compare with another run

Important UX change

A run row should not just be data. It should feel like an object you can operate.

4.3 Workbench

This is the most important screen. It should be the heart of the system.

This is the evolved version of your current `assembly.tsx`.

Purpose

Complete operation of a single run from one place.

Layout

Header zone

- run title
- status chip
- run ID
- created by / started at
- duration
- quick actions
- stage progress bar
- gate state summary

Main workbench tabs

- Overview

- Pipeline
- Intake
- Gates
- Artifacts
- Config
- Activity
- Compare

Your current tabs are close, but they need to be richer.

Tab: Overview

Shows:

- run summary
- current stage
- current gate state
- key warnings
- recent activity
- quick access to outputs
- pipeline mini map

Tab: Pipeline

This is the upgraded version of your current pipeline timeline.

Should include:

- full horizontal or vertical stage rail

- stage duration
- stage owner/agent
- stage inputs and outputs
- gate tied to each relevant stage
- click any stage to inspect details
- rerun stage action
- jump to evidence

This should feel like a mission chain, not a plain status list.

Tab: Intake

Use the current intake data, but present it much better.

Should include:

- section progress cards
- structured grouped responses
- editable mode
- missing/weak fields highlighted
- validation state
- “relaunch from intake” action

Tab: Gates

This needs to become a real operator tool.

Should include:

- gate list

- pass/fail state
- predicates / rule checks
- evidence refs
- reason codes
- fix guidance panel
- re-evaluate gate action

This is where the suite stops being pretty and becomes powerful.

Tab: Artifacts

This should open directly into the artifact explorer for the run.

Should include:

- artifact tree
- preview pane
- metadata
- rendered view
- raw JSON/code view
- download options
- compare artifact versions
- pin important artifacts

Tab: Config

- stage order

- gate map
- standards snapshot
- template snapshot
- environment/config settings
- locked inputs and pins

Tab: Activity

- run events
- status transitions
- stage transitions
- operator actions
- mutation history
- export history

Tab: Compare

- compare this run to prior run
- artifact diffs
- intake diffs
- gate diffs
- changed config
- changed templates/standards used

4.4 New Run page

Your current intake wizard is useful, but it needs to feel more premium and more operational.

Purpose

Create a run with confidence and speed.

Layout

Left step navigator

- Project
- Category
- Intent
- Routing
- Design
- Functional
- Data
- Integrations
- NFR
- Auth
- Final Review

Center form canvas

Clean, spacious form area with better section styling.

Right live summary panel

- completion %
- missing required fields
- warnings
- inferred configuration
- selected templates/standards preview
- launch readiness

UX improvements

- autosave draft
- section completeness indicators
- sticky next/previous actions
- live validation
- quick copy from previous run
- launch summary before submission

This should feel like configuring a machine, not filling out bland admin forms.

4.5 Artifact Explorer

Your current artifact browser is functional but basic.

Purpose

Full control over run outputs and generated files.

Layout

Left pane

Artifact tree / directories.

Center pane

File preview.
Support:

- JSON pretty view
- markdown render
- plain text/code view
- side-by-side diff
- schema/metadata overlays

Right inspector

- file path
- artifact type
- origin stage
- generated at
- version/hash
- linked evidence
- export actions

Functional additions

- pin/favorite
- recent files

- compare versions
- quick open related artifact
- open source stage
- export selected artifacts only

4.6 Knowledge area

Right now the library pages are numerous but feel scattered.

Purpose

Central reference system for the suite.

New structure

Collapse the many library pages into a stronger navigation model.

Knowledge home

- search everything
- categories
- recently viewed docs
- pinned docs
- changed docs
- doc health

Knowledge categories

- System

- Orchestration
- Gates
- Policy
- Canonical
- Standards
- Templates
- Planning
- Verification
- Kit
- Telemetry
- Audit
- Intake

Each category gets:

- summary header
- status
- doc count
- changed recently
- coverage health
- drill-down list

This makes the library feel curated instead of just exposed.

4.7 Health page

Your current health page should become a real system diagnostics surface.

Purpose

System confidence and operational monitoring.

Layout

Health summary band

- overall status
- degraded components
- last full check
- dependency health
- environment state

Diagnostics cards

- stage engine health
- gate engine health
- template resolution health
- artifact system health
- export health
- storage/filesystem health

Trend area

- failure trend

- average run duration
- gate failure frequency
- packaging success rate

Action area

- run diagnostic
- verify contracts
- inspect anomalies
- open maintenance mode

4.8 Logs page

The current logs experience should become a proper operator console.

Purpose

Debugging, tracing, investigation.

Layout

Top controls

- search
- severity filter
- run filter
- stage filter

- time range
- live tail toggle

Main panel

High-density log stream with:

- color coded severity
- timestamps
- source
- run links
- stage links
- expand details

Right inspector

- structured payload
- related run
- related stage
- related gate
- copy/export event

This should feel closer to a real observability console.

4.9 Maintenance Center

This should feel like a restricted engineering bay.

Purpose

Controlled system upkeep, validation, and recovery.

Modules

- verification scan
- drift detection
- coverage audit
- template integrity scan
- standards refresh check
- snapshot/restore
- cleanup tools

Each maintenance action should show

- purpose
- risk
- required inputs
- output
- action button
- last result
- history

This page should feel serious and high-control.

4.10 Export page

The export page should feel like a packaging and deployment dock.

Purpose

Produce deliverables cleanly.

Layout

- latest packages
- export presets
- output type selector
- package manifest preview
- download queue
- export history

Actions

- export kit
- export docs only
- export proof/evidence only
- export snapshot
- export selected artifacts

---

5. Shared component system to build first

Before redesigning pages, the UI needs a reusable premium component layer.

Must-have components

- app shell
- grouped sidebar
- top command bar
- metric cards
- glass panels
- status chips
- stage rail
- gate result cards
- activity timeline
- inspector panel
- split pane layout
- command palette
- empty state blocks
- premium table
- artifact tree
- log stream viewer
- diff viewer
- quick action dock

Without this layer, page redesign will drift.

---

6. Visual language rules

Base look

- dark mode first
- obsidian / graphite background
- glass and metal feel
- subtle blur panels
- thin luminous borders
- strong typography hierarchy
- restrained accent glows

Accent usage

- cyan/blue = live/process
- green = success
- amber = warning
- red = failure
- violet = intelligence/special state

Texture

- faint grid
- subtle radial glow behind key areas
- soft depth, not noisy cyberpunk clutter

The goal is **premium lab**, not gamer dashboard.

---

7. Interaction rules

Motion

Use motion for:

- route transitions
- opening drawers
- selecting a run
- stage state changes
- notification arrivals
- command confirmations

Density

Allow 2 workspace densities:

- Focus mode
- Dense operator mode

Keyboard support

Need:

- command palette
- quick search
- next/previous run
- jump to logs

- jump to artifacts
- rerun selected stage

This is part of the Tony Stark feeling too: fast control.

---

8. Suggested implementation order for `App/`

Phase A — Shell

- redesign sidebar
- add grouped nav
- add top command bar
- add global visual system
- create inspector pattern

Phase B — Command Center

- rebuild dashboard
- add status cards
- live activity
- alerts
- quick actions

Phase C — Runs + Workbench

- rebuild assemblies list into Runs

- evolve assembly page into Workbench
- add tabs for Gates and Activity
- improve stage rail and actions

Phase D — Artifact + Logs + Health

- build artifact explorer
- rebuild logs console
- rebuild health diagnostics

Phase E — Knowledge + Maintenance + Export

- unify library experience
- build maintenance center
- improve export dock

---

9. Final product identity

When this overhaul is done, `App/` should feel like:

AXION Lab OS

- command center
- build intelligence console
- run orchestration station
- artifact and proof workbench
- long-term operating environment

Not:

- a plain admin panel
- a doc browser
- a dev utility page set

Yes:

- a beautiful operational system you actually want open all day

Routes

AXION Test Suite

Route-by-Route UI Overhaul Map (App/src)

1. Current High-Level Structure (Observed)

Your UI currently behaves like a **dev admin panel**.

Main screens include:

- Dashboard
- New Assembly
- Assembly Detail
- Files
- Export
- Health
- Logs
- Features
- Doc Inventory
- Library pages

These will be reorganized into **AXION Lab OS modules**.

---

2. New Top-Level Routes

Current	New

dashboard	command-center
new-assembly	new-run
assembly	workbench
files	artifacts
health	system/health
logs	system/logs
export	export
features	intelligence/features
doc inventory	intelligence/docs
libraries	intelligence/library

---

3. New Folder Structure

Inside `App/src` the structure should evolve to something like:

```
src/

app/
  layout/
    AppShell.tsx
    Sidebar.tsx
    Topbar.tsx
    InspectorPanel.tsx

  pages/
    command-center/
    runs/
    workbench/
    new-run/
    artifacts/
    export/

    intelligence/
```

```
      features/
      docs/
      library/

    system/
      health/
      logs/
      maintenance/
      telemetry/

components/
  cards/
  panels/
  tables/
  pipeline/
  artifacts/
  logs/
  layout/
  forms/

hooks/
lib/
types/
```

This separation is critical so the UI doesn’t turn into one giant component mess.

---

4. Page-by-Page Transformation

4.1 Dashboard → Command Center

Current

Basic metrics and simple navigation.

New

Mission control dashboard.

New Components

CommandCenterPage
  HeroStatusBar
  SystemMetricsGrid
  ActiveRunsPanel
  AlertsPanel
  ActivityTimeline
  QuickActionPanel
  RecentArtifactsStrip

Keep from current code

- metric cards
- run counts
- recent assemblies logic

Replace

- flat layout
- dev styling
- simple cards

4.2 Assemblies List → Runs Page

Current

List of assemblies.

New

Fleet management console.

New Components

RunsPage

RunsFilters
RunsTable
RunsCardGrid
RunInspectorPanel

Table Columns

Run Name
Status
Stage
Gate
Started
Duration
Artifacts
Operator
Actions

Key Actions

Open Workbench
Resume Run
Rerun
Export
Delete
Compare

4.3 Assembly Page → Workbench

This becomes the **most important UI surface in the system**.

Current

Assembly detail with pipeline timeline.

New

Full operational console.

Layout

WorkbenchPage
RunHeader
StageProgressRail

WorkbenchTabs

Tabs

Overview
Pipeline
Intake
Gates
Artifacts
Config
Activity
Compare

Components

RunHeader
StageRail
StageCard
GateStatusCard
ArtifactPreview
ActivityTimeline
RunSummaryPanel

4.4 New Assembly → New Run Wizard

Current

Plain intake form.

New

Guided configuration wizard.

Layout

NewRunPage
  StepSidebar
  IntakeForm
  LiveSummaryPanel

Steps

Project
Category

Intent
Routing
Functional
Data
Integrations
NFR
Auth
Review

Key Features

- progress tracking
- autosave
- validation warnings
- template preview
- launch confirmation

4.5 Files → Artifact Explorer

Current

File browser.

New

Artifact control system.

Layout

```
ArtifactExplorerPage
  ArtifactTree
  FilePreviewPane
  ArtifactInspector
```

Preview Types

JSON

Markdown
Text
Schema
Diff

Inspector

origin stage
timestamp
artifact type
hash
related evidence
download options

4.6 Health → System Health

Current

basic health page.

New

Diagnostics console.

Layout

HealthPage
  SystemHealthSummary
  DiagnosticsGrid
  TrendCharts
  HealthActions

Diagnostics

pipeline engine
gate engine
template resolution
artifact generation
export system
storage

4.7 Logs → Observability Console

Current

basic log stream.

New

operator log console.

Layout

```
LogsPage
  LogFilters
  LogStream
  LogInspector
```

Filters

severity
run
stage
gate
time
source

Features

- live tail
- structured payload
- copy event
- jump to run

4.8 Libraries → Knowledge System

Currently scattered across many pages.

Merge Into

KnowledgeHome
KnowledgeCategory
KnowledgeDocViewer

Categories

System
Orchestration
Gates
Policy
Canonical
Standards
Templates
Planning
Verification
Kit
Telemetry
Audit
Intake

4.9 Export Page

Current

basic export screen.

New

package dock.

Layout

ExportPage
  ExportQueue
  ExportOptions
  PackageManifestPreview
  ExportHistory

Export Types

full kit
docs only

proof only
artifacts only
snapshot

4.10 Maintenance Center (NEW)

This doesn't exist yet but should.

Layout

MaintenancePage
  DiagnosticsActions
  DriftScanner
  ContractVerifier
  CoverageAudit
  SnapshotManager

---

5. Core Component Library to Build First

Before touching pages, build a **premium UI component layer**.

Layout

AppShell
Sidebar
Topbar
InspectorPanel
SplitPane

Data Components

MetricCard
StatusChip
StageRail
GateCard
ArtifactTree
ActivityTimeline
DiffViewer
LogViewer

Control Components

CommandPalette
ActionDock
LaunchModal
InspectorDrawer
AlertTray

---

6. Visual System

Base

obsidian background
glass panels
thin neon borders
soft gradients
depth shadows

Status Colors

blue → processing
green → success
amber → warning
red → failure
violet → intelligence

Typography

large headers
tight system labels
monospace for technical fields

---

7. New Global System Features

Command Palette

Shortcut:

Ctrl + K

Actions:

Start Run

Open Workbench
Search Artifacts
Open Logs
Export Kit
Run Diagnostics

Global Search

Search across:

runs
artifacts
docs
logs
templates

Activity Timeline

Central feed showing:

run started
stage completed
gate failed
artifact generated
export completed
maintenance run

---

8. UX Improvements Over Current System

Faster workflows

Instead of:

open assembly
scroll
find pipeline
open logs

New flow:

open run
click failed stage
logs appear instantly
rerun stage

Artifact workflow

Instead of:

download files
inspect locally

New:

preview
diff
trace stage origin
export

---

9. Final Result

After overhaul, `App/` becomes:

AXION Lab OS

Not:

dev dashboard

But:

mission control
run orchestration system
artifact lab
diagnostics console
build intelligence station

Workbench UI Layout Blueprint

Workbench UI Layout Blueprint

This is the core screen.
If the overhaul is done right, this page becomes the **main lab console** you live in.

The goal is for `Workbench` to feel like:

- command center
- engineering bay
- live system console
- run control surface
- artifact and gate inspection station

Not just a detail page.

---

1. Workbench purpose

The Workbench is the **single-run operating environment**.

From here you should be able to:

- understand the run instantly
- see exactly where it is in the pipeline
- inspect failures
- inspect artifacts
- inspect logs
- inspect intake/config

- rerun stages
- export outputs
- compare against prior runs
- monitor activity without leaving the screen

This should be the page you stay on most of the time.

---

2. Workbench layout model

Use a **3-zone workstation layout**.

2.1 Left zone

Persistent structural navigation for the run.

2.2 Center zone

Main operational canvas.

2.3 Right zone

Context inspector.

This gives the screen depth and makes it feel like a real tool, not a flat page.

---

3. High-level page structure

```
┌──────────────
```

```
┌──────────────────── Left Rail ────────────────────┬──────────── Main Work Area
──────────────────────┬──────────── Right Inspector ────────────┐
│ Run Nav                    │ Run Header                  │ Context Inspector          │
│ - Overview                 │ Pipeline Strip              │ - selected stage           │
│ - Pipeline                 │                             │ - selected artifact        │
│ - Intake                   │ Active Tab Content          │ - selected gate            │
│ - Gates                    │                             │ - warnings                 │
│ - Artifacts                │                             │ - quick actions            │
│ - Config                   │                             │                            │
│ - Activity                 │                             │                            │
│ - Compare                  │                             │                            │
└────────────────────────────────────────────────────────┴────────────────────────────
──────────────────────────┴──────────────────────────────────────────────┘
```

---

4. Top command bar

This should stay consistent across the app, but the Workbench version should feel richer.

Contents

- breadcrumb path
- run title quick search
- active environment chip
- quick actions
- notifications
- user/system menu

Workbench-specific actions

- rerun current stage
- pause/resume run

- export run
- compare run
- open logs
- open artifacts
- open maintenance actions

Feel

- sleek
- thin
- high signal
- no bulky controls

---

5. Run header zone

This is the hero area of the Workbench.

It should immediately answer:

- what run is this
- what state is it in
- what stage is active
- whether something needs attention
- what can I do next

Layout

```
┌──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┐
│ Run Name / ID                                                                                                                  │
│ Status Chip   Stage   Gate State   Started   Duration   Operator                                                               │
│                                                                                                                                │
│ Quick Actions: [Resume] [Rerun Stage] [Open Logs] [Export] [Compare]                                                           │
└──────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────────┘
```

Elements

Primary identity

- run name
- run ID
- project/category badge
- created date
- environment badge

Status strip

- overall run status
- current stage
- active gate
- duration
- progress %
- health state

Quick actions

- rerun stage

- resume run
- inspect failure
- open related artifact
- export outputs

This zone should feel like the control console of the run.

---

6. Pipeline strip

Directly under the header should be a **premium stage rail**.

This is one of the most important visual features.

Purpose

Make the pipeline readable at a glance.

Stage presentation

Each stage should show:

- stage name
- state
- duration
- gate indicator if applicable
- evidence count
- issue badge if failed/warned

Visual states

- idle
- queued
- running
- passed
- warned
- failed
- blocked

Interaction

Clicking a stage should:

- update right inspector
- open stage details in center subpanel or drawer
- expose actions like rerun or inspect evidence

Layout style

Use a horizontal rail for premium feel, but support overflow or condensed mode.

[S0]──[S1]──[S2]──[S3]──[S4]──[S5]──[S6]──[S7]──[S8]──[S9]──[S10]

But each node should feel like a real unit, not a tiny circle.

Better:

- rectangular nodes
- glowing edge on active stage
- pulse for running
- red edge for failure

- subtle filled progress line

---

7. Left run rail

This is not the app sidebar.
This is the **run-local navigation**.

Items

- Overview
- Pipeline
- Intake
- Gates
- Artifacts
- Config
- Activity
- Compare

Optional extras

- Evidence
- Logs
- Notes
- Exports

Behavior

- sticky while scrolling
- icons + labels
- active tab glow
- badges for issues

Example:

- Gates `(1 fail)`
- Artifacts `(14)`
- Activity `(6 new)`

This makes the run feel alive and information-rich.

---

8. Right inspector

This is what gives the Workbench that premium “lab console” feeling.

The right inspector changes based on what is selected.

Possible contexts

- selected stage
- selected gate
- selected artifact
- selected warning
- selected activity event

Example: stage selected

Show:

- stage ID
- status
- started/ended
- duration
- inputs
- outputs
- related artifacts
- related logs
- rerun action

Example: gate selected

Show:

- gate ID
- pass/fail
- rule summary
- failed predicates
- evidence refs
- reason codes
- re-evaluate action

Example: artifact selected

Show:

- artifact type
- path
- origin stage
- timestamp
- size/hash
- open preview
- compare version
- export file

This inspector should reduce page hopping.

---

9. Main content tabs

The center canvas changes based on the active tab.

---

10. Tab 1 — Overview

This is the default landing tab.

Goal

Immediate comprehension.

Layout

Use a dashboard style inside the run.

Row 1

- run summary card
- active stage card
- gate health card
- artifact count card

Row 2

- mini pipeline map
- alerts and blockers
- recent activity feed

Row 3

- latest artifacts
- recent logs/errors
- config summary

What it should tell you

- what is happening
- what is blocked
- what changed
- where to go next

This tab should make you feel oriented instantly.

---

11. Tab 2 — Pipeline

This tab is for deep execution analysis.

Layout

Main area:

- full pipeline rail or vertical detailed stage list

Supporting sections:

- stage detail cards
- stage IO viewer
- gate checkpoints
- stage evidence links

Each stage detail should show

- stage purpose
- inputs consumed
- outputs produced
- validations performed
- gate attached
- logs count
- artifacts produced
- rerun eligibility

Key actions

- rerun stage
- inspect logs
- inspect artifacts
- compare stage outputs to prior run

This is where the system feels serious and powerful.

---

12. Tab 3 — Intake

This should not look like a dead form dump.

Layout

Left

Section navigator:

- Project
- Intent
- Requirements
- Data
- Auth
- Integrations
- NFR
- Constraints

Center

Structured intake responses.

Right

Validation and readiness summary.

Features

- highlight missing fields
- flag weak inputs
- edit intake in place
- show normalized interpretation
- show what templates/standards this intake triggered

This is important because intake is operational input, not just archive data.

---

13. Tab 4 — Gates

This should be one of the strongest tabs.

Layout

Split screen:

- left: gate list
- center: gate detail
- right: evidence / reasons / actions

Gate list should show

- gate ID
- status
- stage attached
- time evaluated
- failure severity
- evidence count

Gate detail should show

- rule description
- predicate-by-predicate result
- expected condition
- actual state
- reason codes
- evidence links

Actions

- re-evaluate gate
- open related logs
- open related artifacts
- rerun source stage

This makes gate failures actually workable.

---

14. Tab 5 — Artifacts

This becomes the embedded artifact lab.

Layout

Three pane:

- left: artifact tree
- center: preview panel
- right: metadata inspector

Preview modes

- rendered markdown
- JSON pretty
- raw code/text
- schema view
- diff mode

Workflow actions

- pin artifact
- compare versions
- trace source stage
- export single artifact
- open related evidence

This should remove the need to leave the Workbench for most artifact tasks.

---

15. Tab 6 — Config

This is the run’s operating contract snapshot.

Show

- stage order
- active pipeline definition
- gate map
- selected templates
- standards snapshot
- risk/policy settings
- environment/config used
- pins and locked inputs

Why it matters

You need to know exactly what rules and configuration produced this run.

This tab makes runs auditable and reproducible.

---

16. Tab 7 — Activity

This should feel like the run’s black box recorder.

Show timeline events for:

- run started
- stage entered
- stage completed
- gate evaluated
- failure triggered
- artifact generated
- export completed
- operator action taken

Filters

- system
- operator
- agent
- stage
- gate
- export

Event details

Selecting one event updates the right inspector.

This is useful for tracing and history without dumping the user into raw logs immediately.

---

17. Tab 8 — Compare

This is a high-value premium feature.

Purpose

Show what changed between runs.

Comparison surfaces

- intake diff
- config diff
- artifact diff
- gate result diff
- stage duration diff
- template/standards diff

Layout

- run selector at top
- comparison summary cards
- detailed diff panes below

This gives the suite a serious engineering feel.

---

18. Supporting embedded modules

These should be reusable across tabs.

18.1 Alerts tray

For current run warnings, blockers, stale status, or export-ready notices.

18.2 Quick action dock

A compact set of actions always available:

- rerun stage
- open logs
- open artifacts
- export
- compare

18.3 Evidence preview drawer

Open evidence without leaving current context.

18.4 Logs mini panel

Show recent stage-specific logs inside Pipeline or Gates tabs.

---

19. Visual design rules for Workbench

Background

- dark obsidian
- soft gradient depth
- faint technical grid texture

Panels

- glassmorphism but restrained

- strong internal spacing
- thin luminous borders
- subtle elevation differences

Typography

- large clear run title
- uppercase micro-labels for system metadata
- monospace for technical IDs and paths
- strong contrast

Accent logic

- cyan for live/process
- green for passed
- amber for warning
- red for fail
- violet for intelligence/special system states

Motion

- subtle stage pulse while running
- smooth tab changes
- inspector slide transitions
- tiny animation on status changes

No flashy sci-fi nonsense.
It should feel expensive, sharp, and useful.

---

20. Density modes

The Workbench should support two display modes.

Focus mode

- larger panels
- more whitespace
- ideal for review and presentations

Operator mode

- denser tables
- more visible data
- faster scanning
- more suitable for heavy daily use

This matters because a lab console should flex to how you work.

---

21. Ideal first-screen experience

When opening a run, within 3 seconds you should know:

- whether it is healthy
- what stage it is on

- whether anything failed
- what artifact was last produced
- what action is most likely next

That is the benchmark.

---

22. Workbench component blueprint

Core components needed:

WorkbenchPage
RunHeader
RunStatusStrip
QuickActionRow
StageRail
StageNode
StageInspectorCard
WorkbenchTabRail
OverviewGrid
GateList
GateDetailPanel
ArtifactTree
ArtifactPreview
ConfigSnapshotPanel
ActivityTimeline
ComparePane
RightInspector
AlertsTray
QuickActionDock
EvidenceDrawer

---

23. Suggested implementation order for Workbench

Pass 1

- new header
- stage rail
- tab structure
- right inspector shell

Pass 2

- overview tab
- pipeline tab
- gates tab

Pass 3

- artifacts tab
- activity tab
- config tab

Pass 4

- compare tab
- quick action dock
- evidence drawer
- polish and motion

That order gives the biggest value fastest.

24. Final intended feeling

When this page is complete, opening a run should feel like entering:

- a premium engineering workstation
- a mission control console
- a lab operating surface with depth and authority

Not a React admin page.

The Workbench should be the screen that makes the entire test suite feel real.

What stays, what evolves, what gets split

What stays, what evolves, what gets split

Keep and evolve

These already have useful value and should be upgraded, not thrown away.

- `pages/assembly.tsx` → becomes the new `WorkbenchPage`
- `components/pipeline-progress.tsx` → becomes the base for a richer `StageRail`
- `components/intake-wizard.tsx` → stays for `New Run`, but gets a premium shell later
- `pages/dashboard.tsx` → becomes `CommandCenterPage`
- `pages/files.tsx` → becomes `ArtifactExplorerPage`
- `pages/health.tsx` → becomes `SystemHealthPage`
- `pages/logs.tsx` → becomes `LogsConsolePage`
- `pages/export.tsx` → becomes `ExportDockPage`
- `pages/maintenance.tsx` → stays, but needs major visual/UX upgrade

Merge into a cleaner Knowledge system

These are currently too fragmented:

- `system-library.tsx`
- `orchestration-library.tsx`
- `gates-library.tsx`

- `policy-library.tsx`
- `intake-library.tsx`
- `canonical-library.tsx`
- `standards-library.tsx`
- `templates-library.tsx`
- `planning-library.tsx`
- `verification-library.tsx`
- `kit-library.tsx`
- `telemetry-library.tsx`
- `audit-library.tsx`

These should eventually sit behind:

- `KnowledgeHomePage`
- `KnowledgeCategoryPage`
- `KnowledgeDocViewer`

Not necessarily all at once, but that is the destination.

Replace outright

These are too structurally limited in their current form:

- `components/app-sidebar.tsx`
- the raw flat shell in `App.tsx`

Those need to become an actual operating shell.

---

2. First files to create

Do this before redesigning individual pages.

2.1 App shell layer

Create these first:

App/src/components/layout/app-shell.tsx
App/src/components/layout/sidebar.tsx
App/src/components/layout/topbar.tsx
App/src/components/layout/page-header.tsx
App/src/components/layout/right-inspector.tsx
App/src/components/layout/section-tabs.tsx

Why first

Right now your app is:

- sidebar
- main
- route content

That is too thin for the kind of system you want.

The shell has to support:

- grouped nav
- top command bar
- page-level headers
- optional right inspector

- premium spacing and depth

Without this, every page redesign will feel disconnected.

---

3. First shared UI primitives to create

Create these next.

App/src/components/ui/status-chip.tsx
App/src/components/ui/metric-card.tsx
App/src/components/ui/glass-panel.tsx
App/src/components/ui/action-button.tsx
App/src/components/ui/empty-state.tsx
App/src/components/ui/toolbar.tsx
App/src/components/ui/inspector-card.tsx
App/src/components/ui/info-grid.tsx

Why

Your current pages are using plain boxes and direct Tailwind markup everywhere.
That makes the redesign drift fast.

These primitives give you:

- consistent premium styling
- reusable panel logic
- cleaner page files
- faster redesign of all pages after Workbench

---

4. Workbench-specific files to create first

This is the real heart of the overhaul.

4.1 New Workbench folder

Create:

App/src/components/workbench/workbench-header.tsx
App/src/components/workbench/workbench-tab-rail.tsx
App/src/components/workbench/stage-rail.tsx
App/src/components/workbench/stage-node.tsx
App/src/components/workbench/stage-detail-card.tsx
App/src/components/workbench/workbench-overview.tsx
App/src/components/workbench/workbench-pipeline.tsx
App/src/components/workbench/workbench-intake.tsx
App/src/components/workbench/workbench-gates.tsx
App/src/components/workbench/workbench-artifacts.tsx
App/src/components/workbench/workbench-config.tsx
App/src/components/workbench/workbench-activity.tsx
App/src/components/workbench/workbench-compare.tsx
App/src/components/workbench/run-summary-strip.tsx
App/src/components/workbench/run-alerts-tray.tsx
App/src/components/workbench/quick-action-dock.tsx

Why this structure

Your current `pages/assembly.tsx` is carrying too much responsibility.
It should become an orchestrator, not the place where every visual and behavioral concern lives.

The page should end up mostly doing:

- fetch data
- map state
- pass data into composed modules

---

5. What `pages/assembly.tsx` should become

Right now `assembly.tsx` is doing too much inline.

Current role

- route handling
- data loading
- mutations
- formatting helpers
- tabs
- actions
- status rendering
- page layout

New role

It should become a slimmer page container:

```
pages/assembly.tsx
 -> fetch run data
 -> fetch config
 -> derive selected tab / selected stage / selected gate
 -> render WorkbenchHeader
 -> render StageRail
 -> render active tab module
 -> render RightInspector
```

Split out from `assembly.tsx`

Move these concerns out first:

Move out immediately

- status badge UI

- header UI
- tab rail UI
- stage rail UI
- overview tab UI
- intake tab UI
- artifacts tab UI

Keep in page temporarily

- route param logic
- query hooks
- mutation hooks
- selection state
- format helpers until utility extraction is worth it

---

6. Exact first split for `assembly.tsx`

This is the safest first pass.

Pass 1 split

Take `assembly.tsx` and extract:

StatusBadge -> components/ui/status-chip.tsx
TABS -> components/workbench/workbench-tab-rail.tsx
pipeline strip area -> components/workbench/stage-rail.tsx
run top summary area -> components/workbench/workbench-header.tsx
overview tab content -> components/workbench/workbench-overview.tsx

Why this order

Because those are the highest visual-value areas and the lowest-risk extractions.

You immediately get:

- cleaner page file
- real component boundaries
- visible premium upgrade fast

---

7. Replace `pipeline-progress.tsx`, don’t delete it

This file is useful, but too limited for the final vision.

Current file

`components/pipeline-progress.tsx`

New role

Use it as the logic seed for:

components/workbench/stage-rail.tsx
components/workbench/stage-node.tsx

What to carry over

- stage list
- stage/gate mapping
- status color logic

- hover interaction idea

What to upgrade

- richer nodes instead of dots
- active stage emphasis
- better labels
- durations
- issue badges
- click-to-inspect
- a premium rail layout

So: **refactor forward**, don't trash it.

---

8. New page route structure to target

You do not need to rename everything on day one, but this is the target structure.

Current important routes

- `/`
- `/new`
- `/assembly/:id`
- `/files`
- `/health`

- `/logs`
- `/docs`
- `/export`
- library routes

Better target

- `/` → command center
- `/runs`
- `/new`
- `/assembly/:id` for now, later maybe `/workbench/:id`
- `/artifacts`
- `/system/health`
- `/system/logs`
- `/system/maintenance`
- `/knowledge`
- `/export`

For now, keep the old URLs if you want less breakage. Just make the UI behave like the new system.

---

9. What to do with `App.tsx` first

`App.tsx` should be the second major refactor after the shell components exist.

Current

It directly renders:

- `AppSidebar`
- `<main>`
- `<Switch>`

Replace with

Something like:

```
<AppShell>
 <Switch>
   ...
 </Switch>
</AppShell>
```

So the first real change in `App.tsx` is

- replace `AppSidebar` usage
- move shell spacing/layout responsibility into `AppShell`
- keep routing mostly intact at first

That gives you a system-wide UI upgrade without needing to rewrite every route immediately.

---

10. New sidebar plan mapped to your current app

Current sidebar problem

It is flat and library-heavy.

Replace with grouped nav

Core Ops

- Command Center
- New Run
- Runs
- Workbench
- Artifacts

Intelligence

- Features
- Knowledge
- Doc Inventory

System

- Health
- Logs
- Maintenance

Output

- Export

Practical note

At first:

- `Runs` can just point to `/`
- `Workbench` is entered from run selection
- `Knowledge` can link to a consolidated page later

So the sidebar can evolve before all routes are rebuilt.

---

11. Workbench tab implementation order

This is the best order for actual build work.

Tier 1

Build first:

- `workbench-header.tsx`
- `stage-rail.tsx`
- `workbench-tab-rail.tsx`
- `workbench-overview.tsx`

Tier 2

Then:

- `workbench-pipeline.tsx`
- `workbench-gates.tsx`

- `right-inspector.tsx`

Tier 3

Then:

- `workbench-artifacts.tsx`
- `workbench-intake.tsx`
- `workbench-activity.tsx`

Tier 4

Then:

- `workbench-config.tsx`
- `workbench-compare.tsx`
- `quick-action-dock.tsx`
- `run-alerts-tray.tsx`

That order gives the biggest visible payoff first while keeping you functional.

---

12. Practical file-by-file order to start

This is the cleanest sequence.

Step 1

Create layout layer:

components/layout/app-shell.tsx
components/layout/sidebar.tsx
components/layout/topbar.tsx
components/layout/right-inspector.tsx

Step 2

Create shared UI primitives:

components/ui/glass-panel.tsx
components/ui/status-chip.tsx
components/ui/metric-card.tsx
components/ui/toolbar.tsx

Step 3

Refactor `App.tsx` to use `AppShell`

Step 4

Create Workbench skeleton:

components/workbench/workbench-header.tsx
components/workbench/workbench-tab-rail.tsx
components/workbench/stage-rail.tsx
components/workbench/workbench-overview.tsx

Step 5

Refactor `pages/assembly.tsx` to use those

Step 6

Add `workbench-pipeline.tsx` and `workbench-gates.tsx`

Step 7

Add right inspector behavior

Step 8

Upgrade artifacts/intake/activity tabs

That is the right order if you want momentum without breaking the app into pieces.

---

13. Files that will probably become much smaller

After the split, these should shrink a lot:

- `pages/assembly.tsx`
- `pages/dashboard.tsx`
- `components/app-sidebar.tsx` → likely removed
- `pages/files.tsx`
- `pages/health.tsx`
- `pages/logs.tsx`

That is a good sign. It means page files are coordinating, not doing everything.

---

14. Final recommendation on where to begin

Start with these exact files first:

App/src/components/layout/app-shell.tsx
App/src/components/layout/sidebar.tsx
App/src/components/layout/topbar.tsx
App/src/components/ui/glass-panel.tsx
App/src/components/ui/status-chip.tsx

App/src/components/workbench/workbench-header.tsx
App/src/components/workbench/workbench-tab-rail.tsx
App/src/components/workbench/stage-rail.tsx
App/src/components/workbench/workbench-overview.tsx

Then refactor:

App/src/App.tsx
App/src/pages/assembly.tsx

That gives you:

- immediate visual identity shift
- a real operating shell
- the beginning of the Tony Stark feel
- the core page upgraded first

Folder Tree

`App/src` Target Folder Tree

```
App/src/
│
├── App.tsx
├── main.tsx
├── index.css
│
├── app/
│   ├── routes/
│   │   ├── route-map.tsx
│   │   ├── route-types.ts
│   │   └── nav-config.ts
│   │
│   ├── layout/
│   │   ├── AppShell.tsx
│   │   ├── Sidebar.tsx
│   │   ├── SidebarGroup.tsx
│   │   ├── SidebarItem.tsx
│   │   ├── Topbar.tsx
│   │   ├── PageHeader.tsx
│   │   ├── RightInspector.tsx
│   │   ├── AppBreadcrumbs.tsx
│   │   └── WorkspaceFrame.tsx
│   │
│   └── providers/
│       ├── theme-provider.tsx
│       ├── command-provider.tsx
│       ├── inspector-provider.tsx
│       └── notification-provider.tsx
│
├── pages/
│   ├── command-center/
│   │   ├── CommandCenterPage.tsx
│   │   ├── sections/
│   │   │   ├── HeroStatusBar.tsx
│   │   │   ├── MetricsGrid.tsx
│   │   │   ├── ActiveRunsPanel.tsx
│   │   │   ├── AlertsPanel.tsx
│   │   │   ├── ActivityPanel.tsx
│   │   │   ├── QuickActionsPanel.tsx
│   │   │   └── RecentOutputsPanel.tsx
```

```
│   │   └── hooks/
│   │       └── use-command-center.ts
│   │
│   ├── runs/
│   │   ├── RunsPage.tsx
│   │   ├── components/
│   │   │   ├── RunsFilters.tsx
│   │   │   ├── RunsToolbar.tsx
│   │   │   ├── RunsTable.tsx
│   │   │   ├── RunsCardGrid.tsx
│   │   │   ├── RunRowActions.tsx
│   │   │   └── RunInspectorPanel.tsx
│   │   └── hooks/
│   │       └── use-runs-page.ts
│   │
│   ├── workbench/
│   │   ├── WorkbenchPage.tsx
│   │   ├── components/
│   │   │   ├── WorkbenchHeader.tsx
│   │   │   ├── RunSummaryStrip.tsx
│   │   │   ├── WorkbenchTabRail.tsx
│   │   │   ├── StageRail.tsx
│   │   │   ├── StageNode.tsx
│   │   │   ├── StageDetailCard.tsx
│   │   │   ├── RunAlertsTray.tsx
│   │   │   ├── QuickActionDock.tsx
│   │   │   ├── WorkbenchOverview.tsx
│   │   │   ├── WorkbenchPipeline.tsx
│   │   │   ├── WorkbenchIntake.tsx
│   │   │   ├── WorkbenchGates.tsx
│   │   │   ├── WorkbenchArtifacts.tsx
│   │   │   ├── WorkbenchConfig.tsx
│   │   │   ├── WorkbenchActivity.tsx
│   │   │   ├── WorkbenchCompare.tsx
│   │   │   ├── GateList.tsx
│   │   │   ├── GateDetailPanel.tsx
│   │   │   ├── ArtifactMiniBrowser.tsx
│   │   │   └── EvidenceDrawer.tsx
│   │   ├── hooks/
│   │   │   ├── use-workbench.ts
│   │   │   ├── use-stage-selection.ts
│   │   │   └── use-workbench-actions.ts
│   │   └── utils/
│   │       ├── workbench-formatters.ts
```

```
│   │       └── workbench-mappers.ts
│   │
│   ├── new-run/
│   │   ├── NewRunPage.tsx
│   │   ├── components/
│   │   │   ├── StepSidebar.tsx
│   │   │   ├── LiveSummaryPanel.tsx
│   │   │   ├── ReviewLaunchPanel.tsx
│   │   │   ├── SectionStatusList.tsx
│   │   │   └── LaunchToolbar.tsx
│   │   └── hooks/
│   │       └── use-new-run.ts
│   │
│   ├── artifacts/
│   │   ├── ArtifactExplorerPage.tsx
│   │   ├── components/
│   │   │   ├── ArtifactTree.tsx
│   │   │   ├── ArtifactPreview.tsx
│   │   │   ├── ArtifactInspector.tsx
│   │   │   ├── ArtifactToolbar.tsx
│   │   │   ├── ArtifactDiffViewer.tsx
│   │   │   └── ArtifactBreadcrumbs.tsx
│   │   └── hooks/
│   │       └── use-artifacts.ts
│   │
│   ├── intelligence/
│   │   ├── features/
│   │   │   └── FeaturesPage.tsx
│   │   ├── docs/
│   │   │   ├── DocInventoryPage.tsx
│   │   │   └── components/
│   │   │       ├── DocInventoryTable.tsx
│   │   │       └── DocInventoryFilters.tsx
│   │   └── knowledge/
│   │       ├── KnowledgeHomePage.tsx
│   │       ├── KnowledgeCategoryPage.tsx
│   │       ├── KnowledgeDocViewerPage.tsx
│   │       ├── components/
│   │       │   ├── KnowledgeSearch.tsx
│   │       │   ├── KnowledgeCategoryGrid.tsx
│   │       │   ├── KnowledgeDocList.tsx
│   │       │   ├── KnowledgeDocViewer.tsx
│   │       │   └── KnowledgeSidebar.tsx
│   │       └── categories/
```

```
│   │       ├── system/
│   │       ├── orchestration/
│   │       ├── gates/
│   │       ├── policy/
│   │       ├── intake/
│   │       ├── canonical/
│   │       ├── standards/
│   │       ├── templates/
│   │       ├── planning/
│   │       ├── verification/
│   │       ├── kit/
│   │       ├── telemetry/
│   │       └── audit/
│   │
│   ├── system/
│   │   ├── health/
│   │   │   ├── HealthPage.tsx
│   │   │   ├── components/
│   │   │   │   ├── SystemHealthSummary.tsx
│   │   │   │   ├── DiagnosticsGrid.tsx
│   │   │   │   ├── TrendPanel.tsx
│   │   │   │   └── HealthActions.tsx
│   │   │   └── hooks/
│   │   │       └── use-health.ts
│   │   │
│   │   ├── logs/
│   │   │   ├── LogsPage.tsx
│   │   │   ├── components/
│   │   │   │   ├── LogFilters.tsx
│   │   │   │   ├── LogStream.tsx
│   │   │   │   ├── LogRow.tsx
│   │   │   │   └── LogInspector.tsx
│   │   │   └── hooks/
│   │   │       └── use-logs.ts
│   │   │
│   │   ├── maintenance/
│   │   │   ├── MaintenancePage.tsx
│   │   │   ├── components/
│   │   │   │   ├── MaintenanceActions.tsx
│   │   │   │   ├── DriftScannerCard.tsx
│   │   │   │   ├── ContractVerifierCard.tsx
│   │   │   │   ├── CoverageAuditCard.tsx
│   │   │   │   └── SnapshotManagerCard.tsx
│   │   │   └── hooks/
```

```
│   │   │       └── use-maintenance.ts
│   │   │
│   │   └── telemetry/
│   │       ├── TelemetryPage.tsx
│   │       └── components/
│   │           ├── TelemetryCharts.tsx
│   │           ├── RunDurationChart.tsx
│   │           ├── GateFailuresChart.tsx
│   │           └── ThroughputPanel.tsx
│   │
│   └── export/
│       ├── ExportPage.tsx
│       ├── components/
│       │   ├── ExportQueue.tsx
│       │   ├── ExportOptions.tsx
│       │   ├── PackageManifestPreview.tsx
│       │   ├── ExportHistory.tsx
│       │   └── ExportToolbar.tsx
│       └── hooks/
│           └── use-export.ts
│
├── components/
│   ├── ui/
│   │   ├── glass-panel.tsx
│   │   ├── status-chip.tsx
│   │   ├── metric-card.tsx
│   │   ├── action-button.tsx
│   │   ├── toolbar.tsx
│   │   ├── inspector-card.tsx
│   │   ├── info-grid.tsx
│   │   ├── section-tabs.tsx
│   │   ├── empty-state.tsx
│   │   ├── loading-state.tsx
│   │   ├── code-pill.tsx
│   │   ├── glow-divider.tsx
│   │   └── density-toggle.tsx
│   │
│   ├── data-display/
│   │   ├── ActivityTimeline.tsx
│   │   ├── KeyValueGrid.tsx
│   │   ├── DetailList.tsx
│   │   ├── DiffViewer.tsx
│   │   ├── JsonViewer.tsx
│   │   ├── MarkdownPreview.tsx
```

```
│   │   └── EmptyPanel.tsx
│   │
│   ├── feedback/
│   │   ├── AlertTray.tsx
│   │   ├── InlineWarning.tsx
│   │   ├── NotificationCenter.tsx
│   │   └── StatusBeacon.tsx
│   │
│   ├── command/
│   │   ├── CommandPalette.tsx
│   │   ├── GlobalSearch.tsx
│   │   ├── QuickActionsMenu.tsx
│   │   └── CommandLauncher.tsx
│   │
│   ├── charts/
│   │   ├── LineMetricChart.tsx
│   │   ├── BarMetricChart.tsx
│   │   └── Sparkline.tsx
│   │
│   └── forms/
│       ├── FieldGroup.tsx
│       ├── FormSection.tsx
│       ├── RequiredBadge.tsx
│       └── ValidationSummary.tsx
│
├── features/
│   ├── runs/
│   │   ├── api/
│   │   │   └── runs-api.ts
│   │   ├── hooks/
│   │   │   ├── use-runs.ts
│   │   │   ├── use-run.ts
│   │   │   └── use-run-actions.ts
│   │   ├── types/
│   │   │   └── run-types.ts
│   │   └── utils/
│   │       └── run-utils.ts
│   │
│   ├── artifacts/
│   │   ├── api/
│   │   │   └── artifacts-api.ts
│   │   ├── hooks/
│   │   │   ├── use-artifact-tree.ts
│   │   │   └── use-artifact-preview.ts
```

```
│   │   └── types/
│   │       └── artifact-types.ts
│   │
│   ├── gates/
│   │   ├── api/
│   │   │   └── gates-api.ts
│   │   ├── hooks/
│   │   │   └── use-gates.ts
│   │   └── types/
│   │       └── gate-types.ts
│   │
│   ├── logs/
│   │   ├── api/
│   │   │   └── logs-api.ts
│   │   └── hooks/
│   │       └── use-log-stream.ts
│   │
│   ├── knowledge/
│   │   ├── api/
│   │   │   └── knowledge-api.ts
│   │   └── hooks/
│   │       └── use-knowledge.ts
│   │
│   └── maintenance/
│       ├── api/
│       │   └── maintenance-api.ts
│       └── hooks/
│           └── use-maintenance-actions.ts
│
├── lib/
│   ├── utils.ts
│   ├── cn.ts
│   ├── format/
│   │   ├── dates.ts
│   │   ├── durations.ts
│   │   ├── numbers.ts
│   │   └── statuses.ts
│   │
│   ├── constants/
│   │   ├── nav.ts
│   │   ├── run-status.ts
│   │   ├── stage-order.ts
│   │   └── gate-status.ts
│   │
```

```
│   └── mappers/
│       ├── run-mappers.ts
│       ├── gate-mappers.ts
│       └── artifact-mappers.ts
│
├── hooks/
│   ├── use-command-palette.ts
│   ├── use-global-search.ts
│   ├── use-density-mode.ts
│   └── use-right-inspector.ts
│
├── types/
│   ├── app.ts
│   ├── ui.ts
│   ├── route.ts
│   └── inspector.ts
│
└── styles/
    ├── theme.css
    ├── utilities.css
    └── motion.css
```

How to think about this tree

`app/`

This is the operating shell layer.
It owns:

- layout
- route map
- app-wide providers
- navigation system

`pages/`

These are route-level screens.
Each page should mostly:

- gather data
- coordinate state
- compose smaller pieces

Not hold tons of inline UI.

`components/`

These are reusable visual building blocks shared across multiple pages.

`features/`

This is where the domain logic starts to live:

- API calls
- feature hooks
- feature types
- feature utilities

This keeps page folders from becoming your data layer too.

`lib/`

Pure helpers, constants, formatting, and mapping.

What this means for your current repo

You do **not** need to create all of this at once.

The first working slice should be:

```
App/src/
├── app/
│   └── layout/
│       ├── AppShell.tsx
│       ├── Sidebar.tsx
│       ├── Topbar.tsx
│       └── RightInspector.tsx
│
├── pages/
│   ├── workbench/
│   │   ├── WorkbenchPage.tsx
│   │   └── components/
│   │       ├── WorkbenchHeader.tsx
│   │       ├── WorkbenchTabRail.tsx
│   │       ├── StageRail.tsx
│   │       └── WorkbenchOverview.tsx
│   │
│   └── command-center/
│       └── CommandCenterPage.tsx
│
├── components/
│   └── ui/
│       ├── glass-panel.tsx
│       ├── status-chip.tsx
│       ├── metric-card.tsx
│       └── toolbar.tsx
│
└── features/
    └── runs/
        ├── hooks/
        └── types/
```

That is the cleanest first slice.

Migration map from your current structure

Move toward these equivalents

- `pages/dashboard.tsx` → `pages/command-center/CommandCenterPage.tsx`

- `pages/assembly.tsx` → `pages/workbench/WorkbenchPage.tsx`
- `pages/files.tsx` → `pages/artifacts/ArtifactExplorerPage.tsx`
- `pages/health.tsx` → `pages/system/health/HealthPage.tsx`
- `pages/logs.tsx` → `pages/system/logs/LogsPage.tsx`
- `pages/export.tsx` → `pages/export/ExportPage.tsx`
- `components/app-sidebar.tsx` → `app/layout/Sidebar.tsx`

Best rule for keeping this clean

Use this rule across the overhaul:

- **route files** = orchestration
- **page components** = screen sections
- **shared components** = reusable UI
- **features** = logic/data by domain
- **lib** = generic helpers only

If you blur those, the app will get messy fast again.

AXION Test Suite UI Overhaul

AXION Test Suite UI Overhaul

Migration Sequence

Phase 0 — Ground rules

Before moving files, follow these rules:

- keep current routes working until replacements are stable
- do not rename everything at once
- build new shells/components beside current ones first
- migrate page-by-page, not system-wide in one sweep
- keep old data hooks alive until new page containers are working
- only delete old files after their replacements are fully wired

The goal is controlled replacement, not a big-bang rewrite.

Phase 1 — Install the new shell without breaking pages

1. Create new layout files first

Create these beside the current structure:

App/src/app/layout/AppShell.tsx
App/src/app/layout/Sidebar.tsx
App/src/app/layout/Topbar.tsx
App/src/app/layout/RightInspector.tsx

App/src/app/layout/PageHeader.tsx

2. Create first shared UI primitives

Create:

App/src/components/ui/glass-panel.tsx
App/src/components/ui/status-chip.tsx
App/src/components/ui/metric-card.tsx
App/src/components/ui/toolbar.tsx

3. Refactor `App.tsx` only enough to use `AppShell`

Do not rebuild routing yet.

What changes

- current route map stays
- page files stay where they are
- old pages render inside the new shell
- current sidebar can be replaced with new grouped sidebar

Result of Phase 1

You get:

- immediate global visual upgrade
- better layout foundation
- minimal risk to existing page behavior

This is the first major win.

Phase 2 — Replace sidebar and navigation behavior

1. Replace `components/app-sidebar.tsx`

Do not mutate it heavily. Retire it cleanly.

New source of truth

Move nav config into something like:

App/src/app/routes/nav-config.ts

2. Sidebar groups to introduce now

Even if routes are still old underneath:

Core Ops

- Dashboard
- New Run
- Runs
- Artifacts

Intelligence

- Features
- Docs
- Libraries

System

- Health

- Logs
- Maintenance

Output

- Export

3. Keep current URLs temporarily

Example:

- `Dashboard` can still point to current `/`
- `New Run` can still point to current `/new`
- `Runs` can point to current dashboard or assemblies view until a dedicated page exists

Result of Phase 2

The app starts feeling like one product instead of a collection of tools.

Phase 3 — Workbench first, because it is the core page

This is the most important migration.

1. Create new workbench folder

Create:

App/src/pages/workbench/WorkbenchPage.tsx
App/src/pages/workbench/components/WorkbenchHeader.tsx
App/src/pages/workbench/components/WorkbenchTabRail.tsx
App/src/pages/workbench/components/StageRail.tsx

App/src/pages/workbench/components/WorkbenchOverview.tsx

2. Keep `pages/assembly.tsx` alive temporarily

Do not delete it yet.

Best approach

Turn `pages/assembly.tsx` into a compatibility wrapper:

- keep the same route
- import and render the new `WorkbenchPage`
- pass through the current params/data flow

That lets you upgrade the page while preserving route stability.

3. First things to extract out of old `assembly.tsx`

In this order:

- header area
- status chips
- tab rail
- pipeline/stage strip
- overview tab content

4. Leave these in old page logic for now

- route params
- queries/fetches
- mutations/actions

- tab state
- selected stage state

Result of Phase 3

You now have the first premium core page without destabilizing the data layer.

Phase 4 — Upgrade the stage system before deeper tabs

1. Use current `components/pipeline-progress.tsx` as the seed

Do not delete it immediately.

Replace gradually with:

App/src/pages/workbench/components/StageRail.tsx
App/src/pages/workbench/components/StageNode.tsx
App/src/pages/workbench/components/StageDetailCard.tsx

2. Move stage display logic into these new files

Keep:

- stage order logic
- stage status mapping
- current progress logic

Add:

- richer nodes

- durations
- issue badges
- click-to-inspect behavior
- inspector integration

3. Connect stage selection to right inspector

This is the first place the right inspector starts earning its keep.

Result of Phase 4

The Workbench begins to feel like a lab console instead of a status page.

Phase 5 — Build Workbench tabs in value order

Do not build every tab at once.

Tier 1 tabs

Build first:

- Overview
- Pipeline
- Gates

Files

WorkbenchOverview.tsx
WorkbenchPipeline.tsx
WorkbenchGates.tsx

These give the highest operational value first.

Tier 2 tabs

Then:

- Artifacts
- Intake
- Activity

Files

WorkbenchArtifacts.tsx
WorkbenchIntake.tsx
WorkbenchActivity.tsx

Tier 3 tabs

Then:

- Config
- Compare

Files

WorkbenchConfig.tsx
WorkbenchCompare.tsx

Important rule

Each tab should become a separate component file.
Do not rebuild the old giant `assembly.tsx` inside the new folder.

Phase 6 — Split feature logic from page logic

Once the new Workbench is visually stable, begin moving logic into feature folders.

Start with runs

Create:

App/src/features/runs/hooks/use-runs.ts
App/src/features/runs/hooks/use-run.ts
App/src/features/runs/hooks/use-run-actions.ts
App/src/features/runs/types/run-types.ts
App/src/features/runs/utils/run-utils.ts

Then move logic out of page files gradually

From `assembly.tsx` / `WorkbenchPage.tsx`, move:

- data mapping
- formatter helpers
- derived status helpers
- run action wrappers

Keep in page:

- composition
- tab selection
- layout concerns

Result of Phase 6

You stop mixing page rendering with domain logic.

Phase 7 — Migrate dashboard into Command Center

After Workbench, do dashboard.

1. Create:

App/src/pages/command-center/CommandCenterPage.tsx

2. Make current `pages/dashboard.tsx` a wrapper

Same strategy as Workbench:

- keep route stable
- render new page component inside it

3. First command center sections

Build only these first:

- Hero status bar
- Metrics grid
- Active runs panel
- Alerts panel

Do not overbuild it on pass one.

Result of Phase 7

The landing page finally matches the new product identity.

Phase 8 — Migrate Files into Artifact Explorer

1. Create:

App/src/pages/artifacts/ArtifactExplorerPage.tsx
App/src/pages/artifacts/components/ArtifactTree.tsx
App/src/pages/artifacts/components/ArtifactPreview.tsx
App/src/pages/artifacts/components/ArtifactInspector.tsx

2. Turn current `pages/files.tsx` into wrapper

Same migration pattern.

3. Pass one features

- artifact tree
- preview pane
- metadata inspector

Leave advanced diffing for later.

Result of Phase 8

Artifact handling becomes operational instead of basic browsing.

Phase 9 — Migrate system pages

Do these after Workbench and Artifacts.

Health

Create:

App/src/pages/system/health/HealthPage.tsx

Logs

Create:

App/src/pages/system/logs/LogsPage.tsx

Maintenance

Create:

App/src/pages/system/maintenance/MaintenancePage.tsx

Strategy

Current pages become wrappers first, then are retired after verification.

Build order

- Health
- Logs
- Maintenance

That order matches operational priority.

Phase 10 — Consolidate library pages into Knowledge system

This should happen later, not early.

Right now the library pages may be ugly, but they are useful.
Do not let this part slow the lab console overhaul.

First create:

App/src/pages/intelligence/knowledge/KnowledgeHomePage.tsx
App/src/pages/intelligence/knowledge/KnowledgeCategoryPage.tsx
App/src/pages/intelligence/knowledge/KnowledgeDocViewerPage.tsx

Then migrate existing library pages behind category views

Do not rewrite all knowledge content at once.

Best migration method

- keep old content/data logic
- render through new knowledge layout
- merge categories progressively

Phase 11 — Clean renames after replacements are stable

Only after the new pages are working should you start retiring old filenames.

Safe rename pattern

Step A

Old route file becomes wrapper:

- `pages/assembly.tsx` renders `WorkbenchPage`
- `pages/dashboard.tsx` renders `CommandCenterPage`

Step B

After stability is proven:

- move route declarations to new page files directly
- remove wrapper layer
- delete old page files

Files likely deleted later

- old `components/app-sidebar.tsx`
- old direct shell layout pieces
- old inline page-heavy versions of dashboard/assembly/files/health/logs

---

Phase 12 — Final route cleanup

Only after pages are stable should you consider route path cleanup.

Keep initially

- `/`
- `/new`
- `/assembly/:id`
- `/files`
- `/health`
- `/logs`

Later target

- `/`
- `/runs`
- `/new`
- `/workbench/:id`
- `/artifacts`
- `/system/health`
- `/system/logs`
- `/system/maintenance`
- `/knowledge`
- `/export`

This should be one of the last steps, not the first.

Exact file handling rules

Files to leave in place at first

Leave these and wrap them later:

- `pages/dashboard.tsx`
- `pages/assembly.tsx`

- `pages/files.tsx`
- `pages/health.tsx`
- `pages/logs.tsx`
- `pages/export.tsx`

Files to create immediately

Create first:

- `app/layout/*`
- `components/ui/*`
- `pages/workbench/*`
- `pages/command-center/*`

Files to replace early

Replace early:

- `components/app-sidebar.tsx`
- any flat shell logic in `App.tsx`

Files to defer

Defer until later:

- library page consolidation
- telemetry page

- compare tab
- advanced artifact diffing
- command palette
- global search

Those are valuable, but not first-pass critical.

Best implementation order in plain sequence

Sequence 1

- create new shell
- create new sidebar
- create UI primitives
- wire `App.tsx` into shell

Sequence 2

- create Workbench folder
- make `pages/assembly.tsx` render new `WorkbenchPage`
- extract header, tabs, stage rail, overview

Sequence 3

- add Pipeline tab
- add Gates tab
- connect right inspector

Sequence 4

- add Artifacts tab
- add Intake tab
- add Activity tab

Sequence 5

- migrate dashboard to Command Center
- migrate files to Artifact Explorer

Sequence 6

- migrate health/logs/maintenance

Sequence 7

- move domain logic into `features/*`
- consolidate knowledge system
- clean routes
- delete wrappers

The safest “do not break the app” pattern

For each major page, use this pattern:

Pattern

1. create new page component in new folder
2. keep old route file
3. old route file imports new page
4. verify UI + data + actions
5. only then retire old implementation

That is the cleanest migration pattern for this overhaul.

Recommended first 10 concrete actions

1. Create `app/layout/AppShell.tsx`
2. Create `app/layout/Sidebar.tsx`
3. Create `app/layout/Topbar.tsx`
4. Create `app/layout/RightInspector.tsx`
5. Create `components/ui/glass-panel.tsx`
6. Create `components/ui/status-chip.tsx`
7. Refactor `App.tsx` to use `AppShell`
8. Create `pages/workbench/WorkbenchPage.tsx`

9. Create `pages/workbench/components/WorkbenchHeader.tsx`

10. Turn `pages/assembly.tsx` into a wrapper for `WorkbenchPage`

That is the cleanest starting line.

file-by-file migration table

Priority 1 — start here

These are the files that set the new shell and the new core experience.

Current file	Action	New destination	Priority	Notes
`App/src/App.tsx`	refactor	`App/src/App.tsx` using `app/layout/AppShell.tsx`	P1	Keep routes working. Only swap in the new shell first.
`App/src/components/app-sidebar.tsx`	replace	`App/src/app/layout/Sidebar.tsx`	P1	Retire this early. It is the clearest “dev tool” visual anchor.
none yet	create	`App/src/app/layout/AppShell.tsx`	P1	New global frame for sidebar, topbar, main canvas, optional right inspector.
none yet	create	`App/src/app/layout/Topbar.tsx`	P1	Adds premium command bar and system identity.
none yet	create	`App/src/app/layout/RightInspector.tsx`	P1	Critical for Workbench depth and stage/gate/

				artifact inspection.
none yet	create	`App/src/components/ui/glass-panel.tsx`	P1	Base premium panel primitive.
none yet	create	`App/src/components/ui/status-chip.tsx`	P1	Centralize all status visuals.
none yet	create	`App/src/components/ui/metric-card.tsx`	P1	Shared for dashboard, health, workbench overview.
none yet	create	`App/src/components/ui/toolbar.tsx`	P1	Shared top-row controls for pages.
`App/src/pages/assembly.tsx`	wrap then migrate	`App/src/pages/workbench/WorkbenchPage.tsx`	P1	Most important page. Keep old route file alive as wrapper first.
`App/src/components/pipeline-progress.tsx`	refactor forward	`App/src/pages/workbench/components/StageRail.tsx` and `StageNode.tsx`	P1	Use as seed, do not delete immediately.
none yet	create	`App/src/pages/workbench/components/WorkbenchHeader.tsx`	P1	First extraction from `assembly.tsx`.
none yet	create	`App/src/pages/workbench/components/WorkbenchTabRail.tsx`	P1	Pull tab UI out of

				`assembly.tsx`.
none yet	create	`App/src/pages/workbench/components/WorkbenchOverview.tsx`	P1	First tab to build.

Priority 2 — core screens right after Workbench

These pages define the overall product feel after the shell is in place.

Current file	Action	New destination	Priority	Notes
`App/src/pages/dashboard.tsx`	wrap then migrate	`App/src/pages/command-center/CommandCenterPage.tsx`	P2	Keep `/` working at first.
`App/src/pages/new-assembly.tsx`	evolve	`App/src/pages/new-run/NewRunPage.tsx`	P2	Good functional base already exists. Keep the wizard logic, improve shell and live summary.
`App/src/components/intake-wizard.tsx`	keep and enhance	`App/src/components/intake-wizard.tsx` or later `pages/new-run/components/*`	P2	Do not rewrite from scratch first.

`App/src/pages/files.tsx`	wrap then migrate	`App/src/pages/artifacts/ArtifactExplorerPage.tsx`	P2	Turn into real artifact lab.
`App/src/pages/health.tsx`	wrap then migrate	`App/src/pages/system/health/HealthPage.tsx`	P2	Good candidate after dashboard and workbench.
`App/src/pages/logs.tsx`	wrap then migrate	`App/src/pages/system/logs/LogsPage.tsx`	P2	Upgrade into operator console.
`App/src/pages/maintenance.tsx`	keep then redesign	`App/src/pages/system/maintenance/MaintenancePage.tsx`	P2	Route can wait; visual/UX upgrade matters more first.
`App/src/pages/export.tsx`	keep then redesign	`App/src/pages/export/ExportPage.tsx`	P2	Lower urgency than Workbench, Dashboard, Files.

Priority 3 — keep functional, migrate later

These matter, but they should not slow the shell and Workbench overhaul.

Current file	Action	New destination	Priority	Notes
`App/src/pages/features.tsx`	keep then reframe	`App/src/pages/intelligence/features/FeaturesPage.tsx`	P3	Can stay mostly intact initially.
`App/src/pages/feature-detail.tsx`	keep then reframe	`App/src/pages/intelligence/features/FeatureDetailPage.tsx` or keep nearby	P3	Depends on how rich the features area becomes.
`App/src/pages/doc-inventory.tsx`	keep then migrate	`App/src/pages/intelligence/docs/DocInventoryPage.tsx`	P3	Useful, but not first-pass critical.

Priority 4 — consolidate into Knowledge system later

These are real content pages, but right now they are fragmented. They should be merged under one knowledge experience later, not immediately.

Current file	Action	New destination	Priority	Notes

`App/src/pages/system-library.tsx`	consolidate later	`App/src/pages/intelligence/knowledge/categories/system/*`	P4	Do not block the main UI overhaul with this now.
`App/src/pages/orchestration-library.tsx`	consolidate later	`.../knowledge/categories/orchestration/*`	P4	Same pattern.
`App/src/pages/gates-library.tsx`	consolidate later	`.../knowledge/categories/gates/*`	P4	Same pattern.
`App/src/pages/policy-library.tsx`	consolidate later	`.../knowledge/categories/policy/*`	P4	Same pattern.
`App/src/pages/intake-library.tsx`	consolidate later	`.../knowledge/categories/intake/*`	P4	Same pattern.
`App/src/pages/canonical-library.tsx`	consolidate later	`.../knowledge/categories/canonical/*`	P4	Same pattern.
`App/src/pages/standards-library.tsx`	consolidate later	`.../knowledge/categories/standards/*`	P4	Same pattern.
`App/src/pages/templates-library.tsx`	consolidate later	`.../knowledge/categories/templates/*`	P4	Same

				pattern.
`App/src/pages/planning-library.tsx`	consolidate later	`.../knowledge/categories/planning/*`	P4	Same pattern.
`App/src/pages/verification-library.tsx`	consolidate later	`.../knowledge/categories/verification/*`	P4	Same pattern.
`App/src/pages/kit-library.tsx`	consolidate later	`.../knowledge/categories/kit/*`	P4	Same pattern.
`App/src/pages/telemetry-library.tsx`	consolidate later	`.../knowledge/categories/telemetry/*`	P4	Same pattern.
`App/src/pages/audit-library.tsx`	consolidate later	`.../knowledge/categories/audit/*`	P4	Same pattern.

Intake page components — keep, don't disturb early

These are already useful structure for the New Run flow. They should be preserved early and only reorganized once the shell and Workbench are stable.

Current file	Action	New destination	Priority	Notes
`App/src/components/intake/page-project.tsx`	keep later	`pages/new-run/components/steps/ProjectStep.tsx`	P3	Functional step component.

`App/src/components/intake/page-category.tsx`	keep	later `.../CategoryStep.tsx`	P3	Keep first.
`App/src/components/intake/page-intent.tsx`	keep	later `.../IntentStep.tsx`	P3	Keep first.
`App/src/components/intake/page-routing.tsx`	keep	later `.../RoutingStep.tsx`	P3	Keep first.
`App/src/components/intake/page-design.tsx`	keep	later `.../DesignStep.tsx`	P3	Keep first.
`App/src/components/intake/page-functional.tsx`	keep	later `.../FunctionalStep.tsx`	P3	Keep first.
`App/src/components/intake/page-data.tsx`	keep	later `.../DataStep.tsx`	P3	Keep first.
`App/src/components/intake/page-integrations.tsx`	keep	later `.../IntegrationsStep.tsx`	P3	Keep first.
`App/src/components/intake/page-auth.tsx`	keep	later `.../AuthStep.tsx`	P3	Keep first.
`App/src/components/intake/page-nfr.tsx`	keep	later `.../NfrStep.tsx`	P3	Keep first.
`App/src/components/intake/page-final.tsx`	keep	later `.../ReviewStep.tsx`	P3	Keep first.
`App/src/components/intake/types.ts`	keep, maybe move later	`App/src/features/runs/types/intake-types.ts` or similar	P3	Move only after run feature layer starts forming.

Shared utility files — leave in place for now

These are not the problem. Don't waste motion moving them too early.

Current file	Action	New destination	Priority	Notes
`App/src/main.tsx`	leave	same	P4	No reason to touch early.
`App/src/index.css`	enhance	same or later split into `styles/*`	P2	Important visually, but can stay where it is initially.
`App/src/lib/queryClient.ts`	leave	same or later `lib/queryClient.ts`	P4	Stable infrastructure.
`App/src/lib/utils.ts`	leave, possibly split later	same or later `lib/format/*`, `lib/mappers/*`	P4	Only split when helpers become crowded.

First new files to create, in exact order

This is the cleanest starting sequence.

1. `App/src/app/layout/AppShell.tsx`
2. `App/src/app/layout/Sidebar.tsx`
3. `App/src/app/layout/Topbar.tsx`
4. `App/src/app/layout/RightInspector.tsx`
5. `App/src/components/ui/glass-panel.tsx`
6. `App/src/components/ui/status-chip.tsx`

7. `App/src/components/ui/metric-card.tsx`
8. `App/src/components/ui/toolbar.tsx`
9. `App/src/pages/workbench/WorkbenchPage.tsx`
10. `App/src/pages/workbench/components/WorkbenchHeader.tsx`
11. `App/src/pages/workbench/components/WorkbenchTabRail.tsx`
12. `App/src/pages/workbench/components/StageRail.tsx`
13. `App/src/pages/workbench/components/WorkbenchOverview.tsx`

First old files to refactor, in exact order

1. `App/src/App.tsx`
2. `App/src/pages/assembly.tsx`
3. `App/src/components/pipeline-progress.tsx`
4. `App/src/pages/dashboard.tsx`
5. `App/src/pages/files.tsx`
6. `App/src/pages/health.tsx`
7. `App/src/pages/logs.tsx`

Files you should not delete early

Do **not** delete these until the replacements are proven live:

- `App/src/pages/assembly.tsx`
- `App/src/pages/dashboard.tsx`
- `App/src/pages/files.tsx`
- `App/src/pages/health.tsx`
- `App/src/pages/logs.tsx`
- `App/src/pages/new-assembly.tsx`
- `App/src/components/pipeline-progress.tsx`
- `App/src/components/intake-wizard.tsx`

They should become wrappers or feed the new version first.

Best migration pattern per file

Use this exact pattern:

- create new component/page in the new destination
- keep the old file
- make the old file render the new one
- verify UI, data, and actions
- only then remove the old inline implementation
- only later rename routes if you still want to

That pattern is what keeps the overhaul controlled.

Phase 1 starter pack

Here's the **Phase 1 starter pack** for the overhaul.

This gives you the first structural layer only:

- new shell
- new sidebar
- new topbar
- right inspector
- first Workbench shell

It is meant to be dropped in **without forcing a full rewrite yet**.

1. Folder slice to create first

```
App/src/
├── app/
│   └── layout/
│       ├── AppShell.tsx
│       ├── Sidebar.tsx
│       ├── Topbar.tsx
│       └── RightInspector.tsx
│
├── components/
│   └── ui/
│       ├── glass-panel.tsx
│       ├── status-chip.tsx
│       ├── metric-card.tsx
│       └── toolbar.tsx
│
└── pages/
    └── workbench/
        ├── WorkbenchPage.tsx
        └── components/
            ├── WorkbenchHeader.tsx
            ├── WorkbenchTabRail.tsx
            ├── StageRail.tsx
            └── WorkbenchOverview.tsx
```

---

2. AppShell.tsx

```
import { ReactNode } from "react";
import { Sidebar } from "./Sidebar";
import { Topbar } from "./Topbar";
import { RightInspector } from "./RightInspector";

type AppShellProps = {
 children: ReactNode;
 inspector?: ReactNode;
};

export function AppShell({ children, inspector }: AppShellProps) {
 return (
   <div className="min-h-screen bg-[#070b12] text-slate-100">
     <div className="pointer-events-none fixed inset-0
bg-[radial-gradient(circle_at_top,rgba(59,130,246,0.12),transparent_30%),radial-gradient(circle_
at_bottom_right,rgba(168,85,247,0.10),transparent_25%)]" />
     <div className="pointer-events-none fixed inset-0 opacity-[0.06]
[background-image:linear-gradient(rgba(255,255,255,0.08)_1px,transparent_1px),linear-gradien
t(90deg,rgba(255,255,255,0.08)_1px,transparent_1px)] [background-size:32px_32px]" />

     <div className="relative grid min-h-screen grid-cols-[280px_minmax(0,1fr)]">
       <Sidebar />

       <div className="grid min-h-screen grid-rows-[72px_minmax(0,1fr)]">
         <Topbar />

         <div className="grid min-h-0 grid-cols-[minmax(0,1fr)_360px] gap-4 p-4">
           <main className="min-w-0 overflow-auto rounded-3xl border border-white/10
bg-white/[0.03] p-4
shadow-[0_0_0_1px_rgba(255,255,255,0.02),0_20px_60px_rgba(0,0,0,0.35)]
backdrop-blur-xl">
             {children}
           </main>

           <aside className="hidden xl:block">
             <RightInspector>{inspector}</RightInspector>
           </aside>
         </div>
       </div>
     </div>
   </div>
```

```
 );
}
```

---

3. `Sidebar.tsx`

```
import { Link, useLocation } from "wouter";
import { cn } from "@/lib/utils";

type NavItem = {
 label: string;
 href: string;
 badge?: string;
};

type NavGroup = {
 title: string;
 items: NavItem[];
};

const NAV_GROUPS: NavGroup[] = [
 {
   title: "Core Ops",
   items: [
     { label: "Command Center", href: "/" },
     { label: "New Run", href: "/new" },
     { label: "Runs", href: "/" },
     { label: "Artifacts", href: "/files" },
   ],
 },
 {
   title: "Intelligence",
   items: [
     { label: "Features", href: "/features" },
     { label: "Doc Inventory", href: "/docs" },
     { label: "Libraries", href: "/system-library" },
   ],
 },
 {
   title: "System",
   items: [
     { label: "Health", href: "/health" },
     { label: "Logs", href: "/logs" },
```

```
    { label: "Maintenance", href: "/maintenance" },
  ],
 },
 {
  title: "Output",
  items: [{ label: "Export", href: "/export" }],
 },
];

export function Sidebar() {
 const [location] = useLocation();

 return (
  <aside className="border-r border-white/10 bg-black/30 px-4 py-5 backdrop-blur-2xl">
    <div className="mb-6 rounded-3xl border border-cyan-400/20 bg-white/[0.04] p-4">
      <div className="mb-1 text-[11px] font-medium uppercase tracking-[0.28em]
text-cyan-300/80">
        AXION
      </div>
      <div className="text-xl font-semibold tracking-tight text-white">
        Lab OS
      </div>
      <div className="mt-2 text-sm text-slate-400">
        Mission control for runs, artifacts, gates, and system state.
      </div>
    </div>

    <nav className="space-y-5">
      {NAV_GROUPS.map((group) => (
        <div key={group.title}>
          <div className="mb-2 px-2 text-[11px] font-medium uppercase tracking-[0.24em]
text-slate-500">
            {group.title}
          </div>

          <div className="space-y-1">
            {group.items.map((item) => {
              const active =
                location === item.href ||
                (item.href !== "/" && location.startsWith(item.href));

              return (
                <Link key={item.href + item.label} href={item.href}>
                  <a
```

```
                    className={cn(
                      "flex items-center justify-between rounded-2xl border px-3 py-2.5 text-sm
transition-all",
                      active
                        ? "border-cyan-400/30 bg-cyan-400/10 text-white
shadow-[0_0_20px_rgba(34,211,238,0.08)]"
                        : "border-transparent bg-white/[0.02] text-slate-300 hover:border-white/10
hover:bg-white/[0.05]"
                    )}
                  >
                    <span>{item.label}</span>
                    {item.badge ? (
                      <span className="rounded-full border border-white/10 px-2 py-0.5 text-[11px]
text-slate-400">
                        {item.badge}
                      </span>
                    ) : null}
                  </a>
                </Link>
              );
            })}
          </div>
        </div>
      ))}
    </nav>
  </aside>
 );
}
```

---

4. Topbar.tsx

```
export function Topbar() {
 return (
   <header className="flex items-center justify-between border-b border-white/10 bg-black/20
px-5 backdrop-blur-xl">
     <div>
       <div className="text-[11px] uppercase tracking-[0.24em] text-slate-500">
         Operating Layer
       </div>
       <div className="text-sm font-medium text-slate-200">
         AXION Test Suite
       </div>
```

```
    </div>

    <div className="flex items-center gap-3">
      <div className="hidden w-[320px] items-center rounded-2xl border border-white/10
bg-white/[0.04] px-3 py-2 md:flex">
        <span className="text-xs text-slate-500">Search runs, artifacts, logs...</span>
      </div>

      <button className="rounded-2xl border border-white/10 bg-white/[0.04] px-3 py-2 text-sm
text-slate-300 hover:bg-white/[0.08]">
        Quick Actions
      </button>

      <div className="rounded-full border border-emerald-400/20 bg-emerald-400/10 px-3 py-1
text-xs font-medium text-emerald-300">
        System Stable
      </div>
    </div>
  </header>
 );
}
```

---

5. RightInspector.tsx

```
import { ReactNode } from "react";

type RightInspectorProps = {
 children?: ReactNode;
};

export function RightInspector({ children }: RightInspectorProps) {
 return (
   <div className="h-full overflow-auto rounded-3xl border border-white/10 bg-white/[0.03] p-4
shadow-[0_20px_60px_rgba(0,0,0,0.35)] backdrop-blur-xl">
     <div className="mb-4">
       <div className="text-[11px] uppercase tracking-[0.24em] text-slate-500">
         Inspector
       </div>
       <h2 className="mt-1 text-lg font-semibold text-white">Context Panel</h2>
       <p className="mt-1 text-sm text-slate-400">
         Stage, gate, artifact, and run context will surface here.
       </p>
```

```
    </div>

    {children ?? (
      <div className="rounded-2xl border border-dashed border-white/10 p-4 text-sm
text-slate-500">
        Select a stage, gate, or artifact to inspect details.
      </div>
    )}
  </div>
 );
}
```

---

6. glass-panel.tsx

```
import { HTMLAttributes } from "react";
import { cn } from "@/lib/utils";

export function GlassPanel({
 className,
 ...props
}: HTMLAttributes<HTMLDivElement>) {
 return (
   <div
     className={cn(
       "rounded-3xl border border-white/10 bg-white/[0.04]
shadow-[0_10px_40px_rgba(0,0,0,0.25)] backdrop-blur-xl",
       className
     )}
     {...props}
   />
 );
}
```

---

7. status-chip.tsx

```
import { cn } from "@/lib/utils";

type StatusTone =
 | "idle"
```

```
 | "running"
 | "success"
 | "warning"
 | "error"
 | "blocked";

type StatusChipProps = {
 label: string;
 tone?: StatusTone;
};

const toneClasses: Record<StatusTone, string> = {
 idle: "border-white/10 bg-white/[0.04] text-slate-300",
 running: "border-cyan-400/20 bg-cyan-400/10 text-cyan-300",
 success: "border-emerald-400/20 bg-emerald-400/10 text-emerald-300",
 warning: "border-amber-400/20 bg-amber-400/10 text-amber-300",
 error: "border-rose-400/20 bg-rose-400/10 text-rose-300",
 blocked: "border-violet-400/20 bg-violet-400/10 text-violet-300",
};

export function StatusChip({
 label,
 tone = "idle",
}: StatusChipProps) {
 return (
   <span
     className={cn(
       "inline-flex items-center rounded-full border px-2.5 py-1 text-xs font-medium",
       toneClasses[tone]
     )}
   >
     {label}
   </span>
 );
}
```

---

8. metric-card.tsx

```
import { ReactNode } from "react";
import { GlassPanel } from "./glass-panel";

type MetricCardProps = {
```

```
 label: string;
 value: ReactNode;
 subtext?: string;
};

export function MetricCard({ label, value, subtext }: MetricCardProps) {
 return (
   <GlassPanel className="p-4">
     <div className="text-[11px] uppercase tracking-[0.22em] text-slate-500">
       {label}
     </div>
     <div className="mt-3 text-3xl font-semibold tracking-tight text-white">
       {value}
     </div>
     {subtext ? (
       <div className="mt-2 text-sm text-slate-400">{subtext}</div>
     ) : null}
   </GlassPanel>
 );
}
```

---

9. `toolbar.tsx`

```
import { ReactNode } from "react";
import { cn } from "@/lib/utils";

type ToolbarProps = {
 left?: ReactNode;
 right?: ReactNode;
 className?: string;
};

export function Toolbar({ left, right, className }: ToolbarProps) {
 return (
   <div
     className={cn(
       "flex flex-col gap-3 rounded-3xl border border-white/10 bg-white/[0.03] p-3 md:flex-row
md:items-center md:justify-between",
       className
     )}
   >
     <div className="min-w-0">{left}</div>
```

```
    <div className="flex flex-wrap items-center gap-2">{right}</div>
   </div>
 );
}
```

---

10. WorkbenchHeader.tsx

```
import { StatusChip } from "@/components/ui/status-chip";
import { GlassPanel } from "@/components/ui/glass-panel";

type WorkbenchHeaderProps = {
 runName: string;
 runId: string;
 status: string;
 stage: string;
 startedAt?: string;
 duration?: string;
};

export function WorkbenchHeader({
 runName,
 runId,
 status,
 stage,
 startedAt,
 duration,
}: WorkbenchHeaderProps) {
 return (
   <GlassPanel className="p-5">
     <div className="flex flex-col gap-4 xl:flex-row xl:items-start xl:justify-between">
       <div>
         <div className="text-[11px] uppercase tracking-[0.24em] text-slate-500">
           Workbench
         </div>
         <h1 className="mt-1 text-2xl font-semibold tracking-tight text-white">
           {runName}
         </h1>
         <div className="mt-2 text-sm text-slate-400">Run ID: {runId}</div>
       </div>

       <div className="flex flex-wrap gap-2">
         <StatusChip label={status} tone="running" />
```

```
        <StatusChip label={`Stage: ${stage}`} tone="idle" />
      </div>
    </div>

    <div className="mt-5 grid gap-3 md:grid-cols-3">
      <div className="rounded-2xl border border-white/10 bg-black/20 p-3">
        <div className="text-[11px] uppercase tracking-[0.18em] text-slate-500">
          Started
        </div>
        <div className="mt-1 text-sm text-slate-200">{startedAt ?? "—"}</div>
      </div>

      <div className="rounded-2xl border border-white/10 bg-black/20 p-3">
        <div className="text-[11px] uppercase tracking-[0.18em] text-slate-500">
          Duration
        </div>
        <div className="mt-1 text-sm text-slate-200">{duration ?? "—"}</div>
      </div>

      <div className="rounded-2xl border border-white/10 bg-black/20 p-3">
        <div className="text-[11px] uppercase tracking-[0.18em] text-slate-500">
          Actions
        </div>
        <div className="mt-2 flex flex-wrap gap-2">
          <button className="rounded-xl border border-white/10 bg-white/[0.04] px-3 py-1.5
text-sm text-slate-300">
            Open Logs
          </button>
          <button className="rounded-xl border border-cyan-400/20 bg-cyan-400/10 px-3 py-1.5
text-sm text-cyan-300">
            Rerun Stage
          </button>
        </div>
      </div>
    </div>
  </GlassPanel>
 );
}
```

---

11. WorkbenchTabRail.tsx

```
import { cn } from "@/lib/utils";
```

```
export type WorkbenchTab =
 | "overview"
 | "pipeline"
 | "intake"
 | "gates"
 | "artifacts"
 | "config"
 | "activity"
 | "compare";

type WorkbenchTabRailProps = {
 activeTab: WorkbenchTab;
 onChange: (tab: WorkbenchTab) => void;
};

const tabs: WorkbenchTab[] = [
 "overview",
 "pipeline",
 "intake",
 "gates",
 "artifacts",
 "config",
 "activity",
 "compare",
];

export function WorkbenchTabRail({
 activeTab,
 onChange,
}: WorkbenchTabRailProps) {
 return (
   <div className="flex flex-wrap gap-2">
     {tabs.map((tab) => {
       const active = tab === activeTab;

       return (
         <button
           key={tab}
           onClick={() => onChange(tab)}
           className={cn(
             "rounded-2xl border px-3 py-2 text-sm capitalize transition-all",
             active
               ? "border-cyan-400/30 bg-cyan-400/10 text-cyan-300"
```

```
            : "border-white/10 bg-white/[0.03] text-slate-400 hover:bg-white/[0.06]
hover:text-slate-200"
          )}
        >
          {tab}
        </button>
      );
    })}
  </div>
);
}
```

---

12. StageRail.tsx

```
import { StatusChip } from "@/components/ui/status-chip";
import { GlassPanel } from "@/components/ui/glass-panel";
import { cn } from "@/lib/utils";

type StageItem = {
 id: string;
 label: string;
 status: "idle" | "running" | "success" | "warning" | "error" | "blocked";
};

type StageRailProps = {
 stages: StageItem[];
 selectedStageId?: string;
 onSelect?: (stageId: string) => void;
};

export function StageRail({
 stages,
 selectedStageId,
 onSelect,
}: StageRailProps) {
 return (
   <GlassPanel className="p-4">
     <div className="mb-3 flex items-center justify-between">
       <div>
         <div className="text-[11px] uppercase tracking-[0.22em] text-slate-500">
           Pipeline
         </div>
```

```
        <div className="mt-1 text-sm text-slate-300">
          Run progression across stages
        </div>
      </div>
    </div>

    <div className="grid gap-3 xl:grid-cols-5">
      {stages.map((stage) => {
        const active = selectedStageId === stage.id;

        return (
          <button
            key={stage.id}
            onClick={() => onSelect?.(stage.id)}
            className={cn(
              "rounded-2xl border p-3 text-left transition-all",
              active
                ? "border-cyan-400/30 bg-cyan-400/10 shadow-[0_0_20px_rgba(34,211,238,0.08)]"
                : "border-white/10 bg-black/20 hover:bg-white/[0.04]"
            )}
          >
            <div className="text-[11px] uppercase tracking-[0.18em] text-slate-500">
              {stage.id}
            </div>
            <div className="mt-1 text-sm font-medium text-white">
              {stage.label}
            </div>
            <div className="mt-3">
              <StatusChip label={stage.status} tone={stage.status} />
            </div>
          </button>
        );
      })}
    </div>
  </GlassPanel>
 );
}
```

---

13. WorkbenchOverview.tsx

```
import { MetricCard } from "@/components/ui/metric-card";
import { GlassPanel } from "@/components/ui/glass-panel";
```

```
export function WorkbenchOverview() {
 return (
   <div className="space-y-4">
     <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
       <MetricCard label="Run Status" value="Running" subtext="Active execution" />
       <MetricCard label="Current Stage" value="S4" subtext="Canonical build" />
       <MetricCard label="Gate State" value="1 Open" subtext="Awaiting evaluation" />
       <MetricCard label="Artifacts" value="12" subtext="Generated so far" />
     </div>

     <div className="grid gap-4 xl:grid-cols-[1.2fr_0.8fr]">
       <GlassPanel className="p-4">
         <div className="text-[11px] uppercase tracking-[0.22em] text-slate-500">
           Run Summary
         </div>
         <div className="mt-3 space-y-3 text-sm text-slate-300">
           <p>
             This panel becomes the main overview surface for current stage,
             blockages, generated outputs, and next recommended actions.
           </p>
           <p>
             For Phase 1, keep it simple and make it visually strong first.
           </p>
         </div>
       </GlassPanel>

       <GlassPanel className="p-4">
         <div className="text-[11px] uppercase tracking-[0.22em] text-slate-500">
           Alerts
         </div>
         <div className="mt-3 rounded-2xl border border-dashed border-white/10 p-4 text-sm
text-slate-500">
           Gate failures, warnings, and high-priority operator notices will surface here.
         </div>
       </GlassPanel>
     </div>
   </div>
 );
}
```

---

14. WorkbenchPage.tsx

```
import { useMemo, useState } from "react";
import { WorkbenchHeader } from "./components/WorkbenchHeader";
import {
 WorkbenchTab,
 WorkbenchTabRail,
} from "./components/WorkbenchTabRail";
import { StageRail } from "./components/StageRail";
import { WorkbenchOverview } from "./components/WorkbenchOverview";

const mockStages = [
 { id: "S0", label: "Init", status: "success" as const },
 { id: "S1", label: "Ingest", status: "success" as const },
 { id: "S2", label: "Validate", status: "success" as const },
 { id: "S3", label: "Resolve", status: "running" as const },
 { id: "S4", label: "Canonical", status: "idle" as const },
];

export function WorkbenchPage() {
 const [activeTab, setActiveTab] = useState<WorkbenchTab>("overview");
 const [selectedStageId, setSelectedStageId] = useState<string>("S3");

 const activeStage = useMemo(
   () => mockStages.find((stage) => stage.id === selectedStageId),
   [selectedStageId]
 );

 return (
   <div className="space-y-4">
     <WorkbenchHeader
       runName="Axion Test Run"
       runId="run_01482"
       status="Running"
       stage={activeStage?.label ?? "Unknown"}
       startedAt="Today, 9:42 AM"
       duration="00:14:22"
     />

     <StageRail
       stages={mockStages}
       selectedStageId={selectedStageId}
       onSelect={setSelectedStageId}
```

```
    />

    <WorkbenchTabRail activeTab={activeTab} onChange={setActiveTab} />

    {activeTab === "overview" && <WorkbenchOverview />}

    {activeTab !== "overview" && (
      <div className="rounded-3xl border border-dashed border-white/10 p-8 text-sm
text-slate-500">
        {activeTab} tab placeholder
      </div>
    )}
  </div>
 );
}
```

---

15. How to wire this in first

`pages/assembly.tsx`

Make the old route file a wrapper:

```
import { WorkbenchPage } from "@/pages/workbench/WorkbenchPage";

export default function AssemblyPage() {
 return <WorkbenchPage />;
}
```

`App.tsx`

Wrap your routes with `AppShell`:

```
import { AppShell } from "@/app/layout/AppShell";
```

Then shape it like:

```
<AppShell>
 <Switch>
   {/* existing routes stay here */}
 </Switch>
```

</AppShell>

That is enough for Phase 1.

---

16. What this gives you immediately

After just this starter pack, the app should already feel:

- darker
- more premium
- more like a system shell
- less like a plain internal tool

And more important, it gives you the right foundation to keep upgrading without redesign drift.

hase 1 integration pass

Here's the **Phase 1 integration pass**.

This is the exact shape for:

- wiring the new shell into `App.tsx`
- keeping current routes alive
- converting `dashboard.tsx` into the first **Command Center** style page without breaking the rest of the app

1. Goal of this pass

At the end of this pass:

- the whole app renders inside the new premium shell
- the old routes still work
- `assembly.tsx` already points to `WorkbenchPage`
- `dashboard.tsx` becomes the first upgraded screen after Workbench
- nothing important gets renamed yet

---

2. `App.tsx` rewrite shape

Do not rebuild routing logic from scratch yet.
Just wrap the current route tree with `AppShell`.

Target structure

```
import { Switch, Route } from "wouter";
import { AppShell } from "@/app/layout/AppShell";

import DashboardPage from "@/pages/dashboard";
```

```
import AssemblyPage from "@/pages/assembly";
import NewAssemblyPage from "@/pages/new-assembly";
import FilesPage from "@/pages/files";
import HealthPage from "@/pages/health";
import LogsPage from "@/pages/logs";
import ExportPage from "@/pages/export";
import FeaturesPage from "@/pages/features";
import FeatureDetailPage from "@/pages/feature-detail";
import MaintenancePage from "@/pages/maintenance";
import DocInventoryPage from "@/pages/doc-inventory";

import SystemLibraryPage from "@/pages/system-library";
import OrchestrationLibraryPage from "@/pages/orchestration-library";
import GatesLibraryPage from "@/pages/gates-library";
import PolicyLibraryPage from "@/pages/policy-library";
import IntakeLibraryPage from "@/pages/intake-library";
import CanonicalLibraryPage from "@/pages/canonical-library";
import StandardsLibraryPage from "@/pages/standards-library";
import TemplatesLibraryPage from "@/pages/templates-library";
import PlanningLibraryPage from "@/pages/planning-library";
import VerificationLibraryPage from "@/pages/verification-library";
import KitLibraryPage from "@/pages/kit-library";
import TelemetryLibraryPage from "@/pages/telemetry-library";
import AuditLibraryPage from "@/pages/audit-library";

function AppRoutes() {
 return (
   <Switch>
     <Route path="/" component={DashboardPage} />
     <Route path="/new" component={NewAssemblyPage} />
     <Route path="/assembly/:id" component={AssemblyPage} />
     <Route path="/files" component={FilesPage} />
     <Route path="/health" component={HealthPage} />
     <Route path="/logs" component={LogsPage} />
     <Route path="/export" component={ExportPage} />
     <Route path="/features" component={FeaturesPage} />
     <Route path="/features/:id" component={FeatureDetailPage} />
     <Route path="/maintenance" component={MaintenancePage} />
     <Route path="/docs" component={DocInventoryPage} />

     <Route path="/system-library" component={SystemLibraryPage} />
     <Route path="/orchestration-library" component={OrchestrationLibraryPage} />
     <Route path="/gates-library" component={GatesLibraryPage} />
     <Route path="/policy-library" component={PolicyLibraryPage} />
```

```
    <Route path="/intake-library" component={IntakeLibraryPage} />
    <Route path="/canonical-library" component={CanonicalLibraryPage} />
    <Route path="/standards-library" component={StandardsLibraryPage} />
    <Route path="/templates-library" component={TemplatesLibraryPage} />
    <Route path="/planning-library" component={PlanningLibraryPage} />
    <Route path="/verification-library" component={VerificationLibraryPage} />
    <Route path="/kit-library" component={KitLibraryPage} />
    <Route path="/telemetry-library" component={TelemetryLibraryPage} />
    <Route path="/audit-library" component={AuditLibraryPage} />
  </Switch>
 );
}

export default function App() {
 return (
  <AppShell>
    <AppRoutes />
  </AppShell>
 );
}
```

---

3. Better `App.tsx` pattern for the inspector

Since the new shell supports a right inspector, give `AppShell` route-aware inspector content later.

For Phase 1, keep it simple.

Phase 1 version

```
export default function App() {
 return (
  <AppShell>
    <AppRoutes />
  </AppShell>
 );
}
```

Phase 1.5 version

Later, once Workbench inspector is ready:

```
export default function App() {
 return (
   <AppShell inspector={<div className="text-sm text-slate-400">No selection yet.</div>}>
     <AppRoutes />
   </AppShell>
 );
}
```

For now, the default placeholder inside `RightInspector` is enough.

---

4. `pages/assembly.tsx` wrapper

You already planned this. This is the exact shape to use now.

```
import { WorkbenchPage } from "@/pages/workbench/WorkbenchPage";

export default function AssemblyPage() {
 return <WorkbenchPage />;
}
```

That keeps `/assembly/:id` working while you rebuild the page.

---

5. `dashboard.tsx` should become a wrapper too

Do the same pattern with dashboard.

New target file

Create:

App/src/pages/command-center/CommandCenterPage.tsx

Then rewrite `pages/dashboard.tsx` as:

```
import { CommandCenterPage } from "@/pages/command-center/CommandCenterPage";

export default function DashboardPage() {
 return <CommandCenterPage />;
}
```

This keeps `/` stable while letting the new page grow cleanly.

---

6. First `CommandCenterPage.tsx` shape

This should be the first upgraded non-Workbench page.

File

`App/src/pages/command-center/CommandCenterPage.tsx`

```
import { MetricCard } from "@/components/ui/metric-card";
import { GlassPanel } from "@/components/ui/glass-panel";
import { Toolbar } from "@/components/ui/toolbar";
import { StatusChip } from "@/components/ui/status-chip";

export function CommandCenterPage() {
 return (
   <div className="space-y-4">
     <Toolbar
       left={
         <div>
           <div className="text-[11px] uppercase tracking-[0.24em] text-slate-500">
             Mission Control
           </div>
           <h1 className="mt-1 text-2xl font-semibold tracking-tight text-white">
             AXION Command Center
           </h1>
           <p className="mt-1 text-sm text-slate-400">
             Live overview of runs, gates, artifacts, and system state.
           </p>
         </div>
       }
       right={
         <>
```

```
          <button className="rounded-2xl border border-white/10 bg-white/[0.04] px-3 py-2
text-sm text-slate-300">
            Review Failures
          </button>
          <button className="rounded-2xl border border-cyan-400/20 bg-cyan-400/10 px-3 py-2
text-sm text-cyan-300">
            Start New Run
          </button>
        </>
      }
    />

    <div className="flex flex-wrap gap-2">
      <StatusChip label="System Stable" tone="success" />
      <StatusChip label="3 Active Runs" tone="running" />
      <StatusChip label="1 Gate Warning" tone="warning" />
    </div>

    <div className="grid gap-4 md:grid-cols-2 xl:grid-cols-4">
      <MetricCard label="Active Runs" value="3" subtext="Currently processing" />
      <MetricCard label="Completed Today" value="12" subtext="Successful run completions" />
      <MetricCard label="Blocked Gates" value="1" subtext="Needs review" />
      <MetricCard label="Artifacts Ready" value="28" subtext="Generated outputs available" />
    </div>

    <div className="grid gap-4 xl:grid-cols-[1.2fr_0.8fr]">
      <GlassPanel className="p-4">
        <div className="text-[11px] uppercase tracking-[0.22em] text-slate-500">
          Active Runs
        </div>
        <div className="mt-4 space-y-3">
          {[
            { name: "Assembler Run A", status: "running", stage: "Resolve" },
            { name: "Assembler Run B", status: "warning", stage: "Validate" },
            { name: "Assembler Run C", status: "success", stage: "Packaged" },
          ].map((run) => (
            <div
              key={run.name}
              className="rounded-2xl border border-white/10 bg-black/20 p-4"
            >
              <div className="flex items-start justify-between gap-3">
                <div>
                  <div className="text-sm font-medium text-white">{run.name}</div>
                  <div className="mt-1 text-xs text-slate-400">
```

```
            Current stage: {run.stage}
          </div>
        </div>
        <StatusChip
          label={run.status}
          tone={
            run.status === "running"
              ? "running"
              : run.status === "warning"
              ? "warning"
              : "success"
          }
        />
      </div>
    </div>
  ))}
 </div>
</GlassPanel>

<div className="space-y-4">
 <GlassPanel className="p-4">
  <div className="text-[11px] uppercase tracking-[0.22em] text-slate-500">
   Alerts
  </div>
  <div className="mt-4 rounded-2xl border border-amber-400/15 bg-amber-400/5 p-4">
   <div className="text-sm font-medium text-amber-300">
    Gate warning detected
   </div>
   <div className="mt-1 text-sm text-slate-400">
    One run requires operator review before continuing.
   </div>
  </div>
 </GlassPanel>

 <GlassPanel className="p-4">
  <div className="text-[11px] uppercase tracking-[0.22em] text-slate-500">
   Recent Activity
  </div>
  <div className="mt-4 space-y-3 text-sm text-slate-300">
   <div className="rounded-2xl border border-white/10 bg-black/20 p-3">
    Artifact bundle exported
   </div>
   <div className="rounded-2xl border border-white/10 bg-black/20 p-3">
    Run entered canonical build
```

```
            </div>
            <div className="rounded-2xl border border-white/10 bg-black/20 p-3">
              Validation completed successfully
            </div>
          </div>
        </GlassPanel>
      </div>
    </div>
  </div>
);
}
```

---

7. What this does for `dashboard.tsx`

It changes the landing page from:

- plain metrics
- flat internal tool feel

into:

- executive status surface
- premium command center
- better launch point for the whole app

And you did it without breaking route stability.

---

8. Minimal nav cleanup for this phase

Your new sidebar already points:

- `/`

- `/new`
- `/files`
- `/features`
- `/docs`
- `/health`
- `/logs`
- `/maintenance`
- `/export`

That is enough for now.

One improvement: make “Runs” not point to `/` long-term.
But for this pass, it is acceptable.

If you want a tiny cleanup now, you can make the label read:

{ label: "Runs Overview", href: "/" }

instead of `"Runs"`.

That avoids the awkward duplicate feeling between Command Center and Runs before a real Runs page exists.

---

9. `index.css` first upgrade layer

You do not need a full theme system yet, but you should give the app better defaults now.

Add or tune your base styles so the new shell looks intentional.

Example direction:

```
:root {
 color-scheme: dark;
}

html,
body,
#root {
 min-height: 100%;
 background: #070b12;
}

body {
 margin: 0;
 color: #e2e8f0;
 font-family: Inter, ui-sans-serif, system-ui, sans-serif;
}
```

If your current CSS already sets some of this, just align it to the darker shell.

---

10. First verification checklist after this pass

After wiring `App.tsx`, `assembly.tsx`, and `dashboard.tsx`, verify:

- app loads inside the new shell
- sidebar navigation still works
- `/` loads the new command center
- `/assembly/:id` loads the new workbench shell
- right inspector appears on xl screens
- no route regressions
- no layout overflow or double-scroll weirdness
- mobile/tablet does not fully break, even if not polished yet

---

11. Best build order for this exact pass

Do it in this order:

1. create `CommandCenterPage.tsx`
2. rewrite `dashboard.tsx` as wrapper
3. refactor `App.tsx` to use `AppShell`
4. verify all current routes still render
5. verify `assembly.tsx` still renders `WorkbenchPage`
6. then tune spacing and CSS

That keeps failures easy to isolate.

---

12. What comes immediately after this

Once this pass is in, the next best move is:

- build `WorkbenchPipeline.tsx`
- build `WorkbenchGates.tsx`
- make `RightInspector` react to selected stage
- then convert `files.tsx` into `ArtifactExplorerPage`

Phase 2 Work Bench

build out the Workbench's real operational depth

In order:

1. `WorkbenchPipeline.tsx`
   Turn the pipeline area from placeholder into a real stage workspace.

2. `WorkbenchGates.tsx`
   Make gate failures and gate state actually inspectable.

3. **connect `RightInspector` to stage selection**
   Clicking a stage should update the inspector with run/stage details.

4. **upgrade `StageRail.tsx`**
   Add richer stage cards:

   - duration
   - issue badge
   - active highlight
   - better status language

5. **replace overview placeholders with live run data**
   The shell is there now, so the next step is making it real.

So the immediate next section is:

Phase 2 — Workbench depth

Files to create next

- `App/src/pages/workbench/components/WorkbenchPipeline.tsx`
- `App/src/pages/workbench/components/WorkbenchGates.tsx`
- `App/src/pages/workbench/components/StageDetailCard.tsx`

Files to update next

- `App/src/pages/workbench/WorkbenchPage.tsx`
- `App/src/app/layout/RightInspector.tsx`
- `App/src/pages/workbench/components/StageRail.tsx`

Goal of this phase

Make the Workbench feel usable, not just styled.

Right now you have:

- shell
- header
- stage rail
- tab rail
- overview structure

What you need next is:

- stage inspection
- gate inspection
- contextual right panel
- real operational flow

WorkbenchPipeline.tsx

WorkbenchPipeline.tsx

Purpose

Make the Pipeline tab the execution control surface for a run.

It should let you:

- see every stage clearly
- inspect stage state
- understand what happened
- move from stage → logs / artifacts / gate context
- prepare for rerun actions later

Layout

Use a **2-column layout** inside the Pipeline tab.

Left

Pipeline stage list or stage stack.

Right

Selected stage detail panel.

Main structure

```
WorkbenchPipeline
├── PipelineToolbar
├── PipelineStageList
│   ├── StageCard
```

```
│   ├── StageCard
│   └── ...
└── StageDetailCard
```

PipelineToolbar

Top strip for quick context.

Show:

- current active stage
- total stages
- passed / failed / blocked counts
- optional filter later

For now keep it simple.

PipelineStageList

This is the main stage inspection area.

Each stage card should show:

- stage ID
- stage name
- status
- duration
- gate attached if any
- artifact count
- warning/failure marker

- whether it is selected

The selected stage should visually stand out.

`StageDetailCard`

When a stage is selected, show:

- stage ID
- full stage name
- current status
- started at
- completed at
- duration
- inputs summary
- outputs summary
- related gate
- related artifacts
- quick actions placeholders

This is the piece that starts making the Workbench feel real.

Data model for first pass

Use a simple local type for now:

```
type PipelineStage = {
 id: string;
```

```
 label: string;
 status: "idle" | "running" | "success" | "warning" | "error" | "blocked";
 duration?: string;
 gate?: string;
 artifacts?: number;
 startedAt?: string;
 finishedAt?: string;
 inputsSummary?: string;
 outputsSummary?: string;
};
```

Files to create now

- `App/src/pages/workbench/components/WorkbenchPipeline.tsx`
- `App/src/pages/workbench/components/StageDetailCard.tsx`

Update next

Update:

- `App/src/pages/workbench/WorkbenchPage.tsx`

So when `activeTab === "pipeline"` it renders the real Pipeline tab instead of placeholder text.

First-pass behavior

- click a stage in the pipeline list
- detail card updates
- selected stage can also be sent to `RightInspector` later

Visual feel

Keep it:

- dark
- premium
- structured
- dense enough to feel useful
- not overloaded

Think:
engineering console panel, not kanban board.

WorkbenchGates.tsx

`WorkbenchGates.tsx`

Purpose

Make the Gates tab the place where gate outcomes are actually workable.

It should let you:

- see all gates tied to the run
- know which gate failed or is blocked
- inspect why
- see supporting evidence
- understand what stage the gate belongs to
- prepare for re-evaluate / rerun actions later

This tab is where the suite stops being just a run viewer and becomes an operator console.

Layout

Use a **3-part layout** inside the Gates tab.

Left

Gate list

Center

Gate detail panel

Right

Evidence / actions / summary panel

For a first pass, center and right can be combined if you want less complexity.

Main structure

```
WorkbenchGates
├── GatesToolbar
├── GatesLayout
│   ├── GateList
│   │   ├── GateRow
│   │   ├── GateRow
│   │   └── ...
│   ├── GateDetailPanel
│   └── GateEvidencePanel
```

GatesToolbar

Top strip for quick state.

Show:

- total gates
- passed count
- warning count
- failed count
- blocked count

Later you can add filtering, but first pass should stay simple.

GateList

This is the navigation area for gate inspection.

Each gate row should show:

- gate ID
- gate name
- attached stage
- status
- severity if failed
- evidence count
- last evaluated time

The selected gate should stand out clearly.

GateDetailPanel

When a gate is selected, show:

- gate ID
- gate name
- attached stage
- status
- severity
- summary description
- evaluation time
- failed predicates
- passed predicates
- reason codes

- notes / interpretation area

This should answer:

- what rule was checked
- what was expected
- what actually happened
- why it failed or warned

`GateEvidencePanel`

Show:

- evidence references
- related artifacts
- related logs
- related stage
- placeholder actions

For first pass, this can be simple cards like:

- `artifact: canonical-spec.json`
- `log: validation warning`
- `stage: S2 Intake Validation`

Later this can become clickable deep links.

Data model for first pass

Use a simple local type:

```
type GateItem = {
 id: string;
 label: string;
 stageId: string;
 status: "success" | "warning" | "error" | "blocked";
 severity?: "low" | "medium" | "high";
 evaluatedAt?: string;
 evidenceCount?: number;
 summary?: string;
 failedPredicates?: string[];
 passedPredicates?: string[];
 reasonCodes?: string[];
 evidence?: string[];
};
```

Files to create now

- `App/src/pages/workbench/components/WorkbenchGates.tsx`

You can keep `GateList`, `GateDetailPanel`, and `GateEvidencePanel` inside that file first if you want speed.

If you want cleaner structure immediately, split them into:

- `GateList.tsx`
- `GateDetailPanel.tsx`

But for first pass, one file is fine.

Update next

Update:

- `App/src/pages/workbench/WorkbenchPage.tsx`

So when `activeTab === "gates"` it renders the real Gates tab.

First-pass behavior

- click a gate in the list
- center detail updates
- evidence panel updates
- failed predicates are clearly visible
- status chips reflect actual tone

Visual feel

This tab should feel:

- precise
- serious
- audit-friendly
- easy to scan

Not noisy, and not overly decorative.

Think:
compliance / validation console inside a futuristic lab UI

Suggested first-pass sections inside the detail panel

Header

- gate ID
- name
- stage
- status chip

Summary block

- one-paragraph explanation

Predicate results

- passed list
- failed list

Reason codes

- compact pill list

Evidence summary

- evidence count
- related items

Connect Workbench selection

Connect Workbench selection to `RightInspector`

Purpose

Make the Workbench feel like a real operating console.

Right now:

- stages live in Pipeline
- gates live in Gates
- inspector exists
- but nothing is connected

This next step makes the screen feel alive.

Goal

When you select something in the Workbench, the right panel should update.

First connected contexts

- selected stage
- selected gate

Later:

- selected artifact
- selected log event

- selected activity item

What changes now

Files to update

- `App/src/app/layout/RightInspector.tsx`
- `App/src/pages/workbench/WorkbenchPage.tsx`
- `App/src/pages/workbench/components/WorkbenchPipeline.tsx`
- `App/src/pages/workbench/components/WorkbenchGates.tsx`

New file to create

- `App/src/pages/workbench/components/InspectorStageCard.tsx`
- `App/src/pages/workbench/components/InspectorGateCard.tsx`

You can combine those into one file at first if you want speed.

---

1. New selection model in `WorkbenchPage.tsx`

The Workbench page should become the source of truth for current selection.

Add selection state

You want state like:

```
type InspectorSelection =
 | { type: "stage"; id: string }
 | { type: "gate"; id: string }
 | null;
```

Then in `WorkbenchPage.tsx`:

```
const [inspectorSelection, setInspectorSelection] =
 useState<InspectorSelection>(null);
```

This is better than letting each tab own inspector behavior separately.

---

2. Pipeline tab should notify selection

`WorkbenchPipeline.tsx` should accept props like:

```
type WorkbenchPipelineProps = {
 stages: PipelineStage[];
 selectedStageId?: string;
 onSelectStage?: (stageId: string) => void;
};
```

When a stage row/card is clicked:

- update local selected stage if needed
- also call `onSelectStage(stage.id)`

That lets the parent Workbench control inspector state.

---

3. Gates tab should notify selection

`WorkbenchGates.tsx` should accept props like:

```
type WorkbenchGatesProps = {
 gates: GateItem[];
 selectedGateId?: string;
```

```
 onSelectGate?: (gateId: string) => void;
};
```

When a gate row is clicked:

- update selected gate
- call `onSelectGate(gate.id)`

---

4. `RightInspector.tsx` should become a real renderer

Right now it is just a shell with placeholder content.

Next step:
let it render based on passed children or passed mode/content.

Simplest version:
keep `RightInspector` as the shell and pass content into it from `AppShell`.

But because `AppShell` wraps routes, the easiest near-term path is:

- keep `RightInspector` as a shell
- pass inspector content from `WorkbenchPage` up through `AppShell` later, or
- for now, render a Workbench-local right column inside the page itself

Best practical move right now

Since routing is already wrapped globally, the fastest clean path is:

temporarily let Workbench render its own inspector content region inside the page body, matching the shell style.

That avoids complicated global route-aware state too early.

So for now:

- keep the app-wide `RightInspector` generic
- add a **Workbench-local inspector card** in the page layout
- later unify it with global inspector plumbing

That is the least messy route.

---

5. Best implementation shape for now

Instead of overcomplicating the shell, change `WorkbenchPage.tsx` layout to:

```
WorkbenchPage
├── Header
├── StageRail
├── TabRail
└── Main Grid
  ├── Active Tab Content
  └── Workbench Inspector Panel
```

That gives you the right-side contextual behavior immediately.

---

6. New layout for `WorkbenchPage.tsx`

After header/rail/tab rail, render:

```
<div className="grid gap-4 xl:grid-cols-[minmax(0,1fr)_320px]">
 <div>{tab content}</div>
 <div>{inspector content}</div>
</div>
```

This gives the page real workstation depth.

---

7. Inspector content rules

If selected item is a stage

Render `InspectorStageCard`

Show:

- stage ID
- stage name
- status
- duration
- started / finished
- gate attached
- artifact count
- short input/output summary
- placeholder quick actions

If selected item is a gate

Render `InspectorGateCard`

Show:

- gate ID
- gate name
- stage attached
- status

- severity
- evidence count
- summary
- reason codes
- failed predicate count
- placeholder quick actions

If nothing selected

Render an idle empty state:

- “Select a stage or gate to inspect”
- maybe show run summary hints

---

8. `InspectorStageCard.tsx`

Purpose

Compact, premium, readable stage context card.

Should include

- top micro-label: `Stage Context`
- stage ID and label
- status chip
- timing block

- gate block
- output block
- action buttons:
    - open logs
    - view artifacts
    - rerun stage

---

9. `InspectorGateCard.tsx`

Purpose

Compact gate analysis card.

Should include

- top micro-label: `Gate Context`
- gate ID and label
- status chip
- stage reference
- severity
- evidence count
- reason code pills
- buttons:
    - view evidence

- open stage
- re-evaluate

---

10. Data flow in `WorkbenchPage.tsx`

Example shape

```
const [selectedStageId, setSelectedStageId] = useState("S3");
const [selectedGateId, setSelectedGateId] = useState<string | null>(null);
const [inspectorSelection, setInspectorSelection] =
 useState<InspectorSelection>({ type: "stage", id: "S3" });
```

On stage click

```
setSelectedStageId(stageId);
setInspectorSelection({ type: "stage", id: stageId });
```

On gate click

```
setSelectedGateId(gateId);
setInspectorSelection({ type: "gate", id: gateId });
```

Then derive:

```
const selectedStage = ...
const selectedGate = ...
```

And render the correct inspector card.

---

11. Behavior rule

Selection should persist when possible.

Example:

- if user clicks a stage in Pipeline tab, inspector shows that stage
- if user switches to Overview, inspector can still show selected stage
- if user clicks a gate in Gates tab, inspector switches to gate
- if user switches tabs again, last selected item can remain until replaced

That makes the whole page feel connected.

---

12. Visual behavior

The inspector should:

- visually match the shell
- stay sticky on larger screens
- update smoothly
- feel like a live technical side console

Not too large, not too empty.

---

13. Minimum first-pass deliverable

For this pass, you only need:

- stage click updates inspector
- gate click updates inspector
- inspector renders stage or gate summary
- empty state when nothing selected

That is enough to make the Workbench feel substantially more powerful.

---

14. After this

Once inspector connectivity is working, the next step is:

Upgrade `StageRail.tsx`

Add:

- durations
- gate badge
- artifact count
- warning/failure indicators
- stronger active-stage visuals

That will make the top pipeline strip feel much more premium and useful.

upgrade

upgrade `StageRail.tsx`

That is the best next move because you already have:

- shell
- workbench
- pipeline tab plan
- gates tab plan
- inspector connection plan

What's missing now is making the **top pipeline strip** feel like a real control surface instead of a simple stage list.

What to add next in `StageRail.tsx`

- stage duration
- gate badge
- artifact count
- warning/failure marker
- stronger active-stage styling
- better status wording
- optional stage subtitle

Result

The Workbench starts feeling much more premium and functional at first glance.

Right now the stage rail tells you basic status.
After this upgrade, it should tell you:

- where the run is
- what happened before
- what has issues
- what stage is selected
- which stages produced outputs

Files to touch next

- `App/src/pages/workbench/components/StageRail.tsx`
- optionally `App/src/pages/workbench/components/StageNode.tsx` if you want to split it cleanly
- `App/src/pages/workbench/WorkbenchPage.tsx` to pass richer stage data

StageRail.tsx upgrade

`StageRail.tsx` upgrade

Purpose

Turn the top stage strip into a real **run command rail**.

Right now it shows:

- stage ID
- stage label
- status

That is enough for scaffolding, but not enough for a premium operator workflow.

After this upgrade, the rail should communicate at a glance:

- which stage is active
- which stages passed cleanly
- which stages have warnings or failures
- how long each stage took
- whether a gate is attached
- whether artifacts were produced
- which stage is selected for deeper inspection

What the upgraded rail should show per stage

Each stage node/card should display:

- stage ID
- stage name
- status chip
- duration
- gate badge if present
- artifact count
- issue marker if warning/error/blocked
- selected state
- active state

Optional later:

- started time
- owner/agent
- rerunnable badge

New stage data shape

Use a richer local type for now.

```
type StageItem = {
  id: string;
  label: string;
  status: "idle" | "running" | "success" | "warning" | "error" | "blocked";
  duration?: string;
  gate?: string;
  artifacts?: number;
  issues?: number;
  subtitle?: string;
};
```

Field meaning

- `duration`: how long the stage ran
- `gate`: gate tied to the stage
- `artifacts`: count produced by this stage
- `issues`: warnings/errors count or just a visible badge count
- `subtitle`: optional micro-context like `validation`, `canonical build`, `template fill`

Layout direction

Keep the current horizontal rail feel, but make each stage tile denser and more informative.

Structure

```
StageRail
├── Rail header
│   ├── title
│   ├── summary text
```

```
│   └── optional counts
└── Stage grid / horizontal strip
    ├── StageNode
    ├── StageNode
    └── ...
```

Rail header

Top area should include:

- `Pipeline`
- short helper text like `Run progression across stages`
- optional run summary:
    - total stages
    - current stage
    - issues count

Example:

- `10 stages`
- `1 running`
- `2 flagged`

This can be simple in pass one.

Stage node visual structure

Each stage node should look like a compact technical card.

Suggested content order

Top row

- stage ID
- selected indicator / active glow

Main label

- stage name
- optional subtitle below it

Meta row

- duration
- gate badge
- artifact count

Bottom row

- status chip
- issue badge if needed

Status language

Use stronger readable language.

Instead of raw codes only, let the node read like:

- Running
- Passed
- Warning
- Failed
- Blocked
- Pending

Stage ID still stays visible, but the human-readable state matters more.

Visual state rules

Selected stage

- brighter border
- cyan glow
- slightly raised card feel

Active/running stage

- pulse or stronger cyan accent

- most visually prominent

Success

- clean, quiet green

Warning

- amber accent, not overwhelming

Error

- red accent, clear but controlled

Blocked

- violet accent

Idle

- subdued neutral

Interaction rules

Clicking a stage should:

- select it
- update Workbench selection
- update inspector
- visually persist selection

Hovering a stage should:

- slightly brighten border/background
- feel responsive, not flashy

Best file structure

You can upgrade in one file first, but cleaner structure is:

- `StageRail.tsx`
- `StageNode.tsx`

Why split

`StageRail` should manage:

- list rendering
- header
- selected ID handling from parent props

`StageNode` should manage:

- individual stage card UI

That keeps the rail clean as it grows.

`StageNode.tsx` responsibilities

It should accept:

```
type StageNodeProps = {
  stage: StageItem;
  active?: boolean;
  selected?: boolean;
  onClick?: () => void;
};
```

And render:

- ID
- label
- subtitle
- duration
- gate badge
- artifact count
- status chip
- issue badge

New UI elements inside stage node

Duration

Use a small muted text block:

- `00:01:42`
- or `1m 42s`

Gate badge

Small pill:

- `G1`
- `G2`
- `No Gate` omitted

Artifact count

Compact text/pill:

- `3 artifacts`

Issue marker

Only show if `issues > 0` or state is warning/error:

- `2 issues`
- or a small red/amber badge

Suggested node layout

```
┌──────────────────────────────────┐
│ S3                 ●   │
│ Standards Resolve        │
│ resolver snapshot        │
│                          │
│ 00:01:42   G3   2 files  │
│ Running        1 issue   │
└──────────────────────────────────┘
```

That is much closer to the lab-console feel.

Update `WorkbenchPage.tsx`

You'll need to pass richer mock data first.

Instead of:

```
const mockStages = [
  { id: "S0", label: "Init", status: "success" },
  ...
];
```

Move to:

```
const mockStages = [
  {
    id: "S0",
    label: "Init",
    subtitle: "run boot",
    status: "success",
    duration: "00:00:08",
    artifacts: 1,
  },
  {
    id: "S1",
    label: "Ingest",
    subtitle: "input normalization",
    status: "success",
    duration: "00:00:33",
    artifacts: 2,
  },
  {
    id: "S2",
    label: "Validate",
    subtitle: "intake checks",
    status: "warning",
    duration: "00:00:27",
    gate: "G1",
    artifacts: 1,
    issues: 1,
```

```
  },
  {
    id: "S3",
    label: "Resolve",
    subtitle: "standards resolution",
    status: "running",
    duration: "00:01:12",
    gate: "G3",
    artifacts: 3,
    issues: 1,
  },
  {
    id: "S4",
    label: "Canonical",
    subtitle: "spec build",
    status: "idle",
    gate: "G2",
    artifacts: 0,
  },
];
```

This alone makes the UI feel much richer.

First-pass build target

For this step, complete only this:

- richer stage card layout
- selected state
- duration
- gate badge
- artifact count
- issue marker
- better status display

Do not overbuild animations or deep actions yet.

What this improves immediately

Once upgraded, the top rail becomes:

- useful at a glance
- more premium visually
- more informative without opening tabs
- a real entry point into stage inspection

This matters because the rail is one of the first things you see on the Workbench.

ArtifactExplorerPage

Next is:

`ArtifactExplorerPage` / `files.tsx` overhaul

This is the next best move because after:

- shell
- workbench structure
- pipeline/gates planning
- inspector flow
- upgraded stage rail

…the next real workflow surface is **artifacts**.

That is where the suite becomes truly usable instead of just monitorable.

Goal

Turn `files.tsx` from a basic file browser into a real **artifact lab**.

What it should support

- artifact tree on the left
- preview pane in the center
- metadata inspector on the right
- file type aware viewing
- trace artifact back to source stage
- export/download actions
- later: diff view

Files to create next

- `App/src/pages/artifacts/ArtifactExplorerPage.tsx`
- `App/src/pages/artifacts/components/ArtifactTree.tsx`
- `App/src/pages/artifacts/components/ArtifactPreview.tsx`
- `App/src/pages/artifacts/components/ArtifactInspector.tsx`

File to update

- `App/src/pages/files.tsx` → wrapper for `ArtifactExplorerPage`

First-pass layout

```
ArtifactExplorerPage
├── ArtifactToolbar
└── 3-column workspace
    ├── ArtifactTree
    ├── ArtifactPreview
    └── ArtifactInspector
```

First-pass preview modes

- JSON
- markdown
- plain text
- simple code/text fallback

Inspector should show

- file name
- path
- artifact type
- origin stage
- timestamp
- size/hash placeholder
- actions:
    - open related stage
    - export artifact
    - view metadata

Why this is next

Because after inspecting a stage or gate, the next thing you naturally want is:

- “show me the output”
- “show me the generated file”
- “show me what the stage produced”

That makes artifacts the next highest-value workflow.

ArtifactExplorerPage cont

`ArtifactExplorerPage`

Purpose

Turn artifacts into a real working surface, not just a storage list.

This page should let you:

- browse generated outputs quickly
- preview files without leaving the app
- inspect metadata
- understand what stage created a file
- move from artifact back to run context
- export what you need

This is the **artifact lab** of the suite.

---

1. Layout

Use a **3-column workstation layout**.

Left

Artifact tree

Center

Artifact preview

Right

Artifact inspector

That gives you a proper operating surface immediately.

---

2. Main structure

```
ArtifactExplorerPage
├── ArtifactToolbar
└── ArtifactWorkspace
    ├── ArtifactTree
    ├── ArtifactPreview
    └── ArtifactInspector
```

---

3. `ArtifactToolbar`

Top strip for navigation and actions.

Show

- page title
- current run or scope if applicable
- search box placeholder
- filter placeholder
- quick actions

Actions

- export selected
- refresh
- open in workbench
- view source stage

For first pass, keep these mostly visual except basic selection-aware buttons if easy.

---

4. `ArtifactTree`

This is the navigation surface.

Purpose

Let the user browse artifact folders and files naturally.

Should show

- folder/file hierarchy
- selected item state
- artifact type hints
- optional counts on folders
- optional stage grouping later

First-pass behavior

- click folder to expand/collapse
- click file to preview
- selected file highlights clearly

First-pass data model

```
type ArtifactNode = {
  id: string;
  name: string;
  type: "folder" | "file";
  path: string;
  children?: ArtifactNode[];
  fileType?: "json" | "md" | "txt" | "yaml" | "unknown";
  stageId?: string;
};
```

---

5. `ArtifactPreview`

This is the main reading surface.

Purpose

Show the content of the selected artifact clearly.

First-pass preview modes

- JSON pretty view
- markdown render
- plain text
- generic code/text fallback

Behavior

If no file is selected:

- show empty state

If folder is selected:

- show helper state like `Select a file to preview`

If file is selected:

- show preview based on type

---

6. `ArtifactInspector`

This is the context panel for the selected artifact.

Show

- file name
- full path
- type
- origin stage
- related run
- timestamp
- size
- hash placeholder
- export action
- open related stage action

Why it matters

This is what makes artifacts feel connected to the rest of the system.

Without this, it is just a file viewer.

---

7. First-pass preview rules

JSON

Pretty print inside a scrollable code-style panel.

Markdown

Render as readable formatted content.

Text / YAML / unknown

Render in monospace block.

You do not need advanced syntax highlighting on pass one.

---

8. Suggested file set

Create:

- `App/src/pages/artifacts/ArtifactExplorerPage.tsx`
- `App/src/pages/artifacts/components/ArtifactTree.tsx`
- `App/src/pages/artifacts/components/ArtifactPreview.tsx`
- `App/src/pages/artifacts/components/ArtifactInspector.tsx`

Then update:

- `App/src/pages/files.tsx`

to become a wrapper.

---

9. `files.tsx` wrapper

Same pattern as the Workbench and Command Center migration.

```
import { ArtifactExplorerPage } from "@/pages/artifacts/ArtifactExplorerPage";

export default function FilesPage() {
  return <ArtifactExplorerPage />;
}
```

That keeps route stability.

---

10. `ArtifactExplorerPage.tsx` responsibilities

This page should own:

- selected node state
- expanded folder state
- mock artifact tree data for first pass
- selected artifact content mapping

It should not try to do every viewer type inline forever, but first pass can be pragmatic.

---

11. Recommended first-pass state

```
const [selectedNodeId, setSelectedNodeId] = useState<string | null>(null);
const [expandedFolderIds, setExpandedFolderIds] = useState<string[]>(["root"]);
```

Then derive:

- selected node
- preview mode
- inspector content

12. Suggested first mock tree

Use something like:

```
const mockArtifacts = [
  {
    id: "root",
    name: "run_01482",
    type: "folder",
    path: "/run_01482",
    children: [
      {
        id: "canonical",
        name: "canonical",
        type: "folder",
        path: "/run_01482/canonical",
        children: [
          {
            id: "canonical-spec",
            name: "canonical-spec.json",
            type: "file",
            path: "/run_01482/canonical/canonical-spec.json",
            fileType: "json",
            stageId: "S4",
          },
        ],
      },
      {
        id: "verification",
        name: "verification",
        type: "folder",
        path: "/run_01482/verification",
        children: [
          {
            id: "gate-report",
            name: "gate-report.md",
            type: "file",
            path: "/run_01482/verification/gate-report.md",
            fileType: "md",
            stageId: "S8",
          },
```

```
      ],
    },
    {
      id: "logs-file",
      name: "run-summary.txt",
      type: "file",
      path: "/run_01482/run-summary.txt",
      fileType: "txt",
      stageId: "S10",
    },
  ],
},
];
```

That is enough to build the experience.

---

13. Inspector data shape

For first pass, simple is enough:

```
type ArtifactMeta = {
  id: string;
  name: string;
  path: string;
  fileType: string;
  stageId?: string;
  size?: string;
  updatedAt?: string;
  hash?: string;
};
```

---

14. Visual rules

Artifact tree

- compact

- clear selection
- folder indentation
- soft hover
- not too busy

Preview pane

- biggest panel on the page
- easy to read
- scrollable
- strong contrast
- premium panel framing

Inspector

- narrower
- metadata-first
- action buttons below metadata

---

15. First-pass page layout

Use something like:

Toolbar
3-column grid:
- 280px tree
- minmax(0, 1fr) preview
- 320px inspector

That will feel like a proper workstation.

---

16. What to keep out of first pass

Do not overbuild these yet:

- side-by-side diff

- advanced syntax highlighting
- inline editing
- version history
- artifact pinning
- breadcrumb drilldown logic beyond basics
- drag/drop

Those can come later.

---

17. What this unlocks

Once this page is in place, the test suite gets a real workflow loop:

- inspect run
- inspect stage
- inspect gate
- open artifact
- preview output
- trace back to source stage

That is a major jump in usefulness.

HealthPage overhaul

`HealthPage` overhaul

Purpose

Turn Health into a real **system diagnostics console** instead of a basic status page.

After Workbench and Artifacts, this is the next important operational surface because it answers:

- is the suite healthy
- what subsystem is degraded
- where risk is building
- what needs attention first

---

1. Layout

Use a **dashboard diagnostics layout**.

```
HealthPage
├── HealthToolbar
├── HealthStatusBand
├── DiagnosticsGrid
├── TrendPanels
└── HealthActionsPanel
```

---

2. `HealthToolbar`

Top strip with:

- page title

- last check time
- environment
- refresh action
- run diagnostics action

Keep it simple on first pass.

---

3. `HealthStatusBand`

This is the hero strip.

Show

- overall system status
- degraded subsystem count
- active incidents
- last successful verification
- operator attention level

This should feel like a mission-critical health banner.

---

4. `DiagnosticsGrid`

This is the core of the page.

Each card should represent a major subsystem:

- pipeline engine
- gate engine
- template resolution
- artifact generation
- export system
- storage/filesystem
- run orchestration
- verification layer

Each card shows

- subsystem name
- status chip
- short summary
- last check time
- issue count
- quick action placeholder

---

5. `TrendPanels`

Show a few useful trend surfaces:

- run success trend
- gate failure trend

- average run duration
- artifact generation success rate

For first pass, simple metric cards or placeholder chart panels are enough.

You do not need real charts immediately if the data is not ready.

---

6. `HealthActionsPanel`

This is the operator action area.

Show actions like

- Run diagnostic
- Verify contracts
- Open maintenance
- View failure logs
- Inspect storage issues

Even if the actions are placeholders first, the page should feel actionable.

---

7. Suggested data model for first pass

```
type HealthSubsystem = {
 id: string;
 label: string;
 status: "success" | "warning" | "error" | "blocked";
 summary: string;
 lastChecked?: string;
 issues?: number;
```

```
};
```

---

8. Files to create

- `App/src/pages/system/health/HealthPage.tsx`
- `App/src/pages/system/health/components/SystemHealthSummary.tsx`
- `App/src/pages/system/health/components/DiagnosticsGrid.tsx`
- `App/src/pages/system/health/components/TrendPanel.tsx`
- `App/src/pages/system/health/components/HealthActions.tsx`

File to update

- `App/src/pages/health.tsx` → wrapper for `HealthPage`

---

9. `health.tsx` wrapper

```
import { HealthPage } from "@/pages/system/health/HealthPage";

export default function LegacyHealthPage() {
 return <HealthPage />;
}
```

---

10. Visual feel

Health should feel:

- precise

- high signal
- calm but serious
- premium enterprise diagnostics

Not loud, not cluttered.

Use:

- green for healthy
- amber for degraded
- red for failing
- violet for blocked/special states

---

11. First-pass priority

Build in this order:

1. `HealthPage.tsx`
2. `SystemHealthSummary.tsx`
3. `DiagnosticsGrid.tsx`
4. `HealthActions.tsx`
5. `TrendPanel.tsx`

That gets the most value fastest.

LogsPage overhaul

`LogsPage` overhaul

Purpose

Turn Logs into a real **observability console** instead of a plain log dump.

After Workbench, Artifacts, and Health, this is the next major operational surface because it answers:

- what happened
- when it happened
- where it happened
- which run / stage / gate it belongs to
- what to inspect next

This page is the system's debugging nerve center.

---

1. Layout

Use a **3-part console layout**.

```
LogsPage
├── LogsToolbar
├── LogsWorkspace
│   ├── LogFilters
│   ├── LogStream
│   └── LogInspector
```

---

2. `LogsToolbar`

Top strip with:

- page title
- live tail toggle
- refresh action
- export logs action
- current scope summary

Scope summary examples

- all runs
- current environment
- selected run
- last 24 hours

For first pass, this can be mostly visual.

---

3. `LogFilters`

This is the control surface for narrowing the stream.

First-pass filters

- severity
- run
- stage
- gate

- source
- time range

You do not need every filter to be deeply wired on pass one, but the structure should be there.

---

4. `LogStream`

This is the main list of events.

Each log row should show:

- timestamp
- severity
- message
- source
- run reference
- stage reference if present
- gate reference if present

Behavior

- click row to inspect full details
- selected row highlights clearly
- rows should be dense but readable
- different severities should be instantly scannable

---

5. `LogInspector`

This is the detail panel for the selected event.

Show

- full timestamp
- severity
- message
- source
- run ID
- stage ID
- gate ID
- structured payload preview
- related actions

Actions

- open run
- open stage
- open artifact
- copy event
- export event

This is what makes logs feel connected to the rest of the suite.

---

6. Suggested data model for first pass

```
type LogItem = {
 id: string;
 timestamp: string;
 severity: "info" | "success" | "warning" | "error";
 message: string;
 source: string;
 runId?: string;
 stageId?: string;
 gateId?: string;
 payload?: Record<string, unknown>;
};
```

---

7. Files to create

- `App/src/pages/system/logs/LogsPage.tsx`
- `App/src/pages/system/logs/components/LogFilters.tsx`
- `App/src/pages/system/logs/components/LogStream.tsx`
- `App/src/pages/system/logs/components/LogRow.tsx`
- `App/src/pages/system/logs/components/LogInspector.tsx`

File to update

- `App/src/pages/logs.tsx` → wrapper for `LogsPage`

---

8. `logs.tsx` wrapper

```
import { LogsPage } from "@/pages/system/logs/LogsPage";
```

```
export default function LegacyLogsPage() {
 return <LogsPage />;
}
```

---

9. `LogFilters`

First-pass controls

- severity pills or select
- run dropdown
- stage dropdown
- time range dropdown
- search field placeholder

You can keep logic shallow first, but layout should feel real.

---

10. `LogStream`

Recommended row content

[10:42:18] [warning] Gate G3 evaluation returned unresolved evidence
source: gate-engine run: run_01482 stage: S3

Each row should have:

- compact structure
- good scan rhythm
- selected state

- hover state

---

11. `LogInspector`

Suggested sections

Header

- severity chip
- timestamp
- source

Message

- full event message

Context

- run ID
- stage ID
- gate ID

Payload

- JSON block or formatted key/value list

Actions

- open related run

- copy JSON
- export entry

---

12. Visual feel

Logs should feel:

- technical
- high-density
- clean
- premium
- easy to scan for problems

Think:
enterprise observability console with AXION styling

Not:

- terminal cosplay
- noisy hacker theme
- plain table dump

---

13. First-pass build order

1. `LogsPage.tsx`
2. `LogStream.tsx`

3. `LogRow.tsx`
4. `LogInspector.tsx`
5. `LogFilters.tsx`

That order gets the main workflow in place first.

---

14. What this unlocks

Once Logs is in place, the operational loop becomes much stronger:

- inspect run in Workbench
- inspect gate or stage
- open artifact
- check Health
- trace exact system events in Logs

That is the point where the suite starts feeling like a real lab operating system.

MaintenancePage overhaul

`MaintenancePage` overhaul

Purpose

Turn Maintenance into the controlled engineering bay of the suite.

This page should feel like the place where you:

- run system checks
- verify contracts
- detect drift
- audit coverage
- manage recovery actions
- perform deliberate upkeep

Not a settings page, and not a generic tools page.

Layout

Use a structured control-room layout:

```
MaintenancePage
├── MaintenanceToolbar
├── MaintenanceStatusBand
├── MaintenanceActionsGrid
├── VerificationAndAuditRow
└── RecoveryAndSnapshotsRow
```

`MaintenanceToolbar`

Top strip with:

- page title
- environment
- last maintenance run
- quick action buttons

First-pass actions:

- Run Diagnostics
- Verify Contracts
- Scan Drift
- Open Logs

MaintenanceStatusBand

Hero summary area showing:

- overall maintenance readiness
- unresolved issues
- drift warnings
- last successful verification
- snapshot availability

This should tell you, immediately, whether the system is safe and stable enough to keep operating.

MaintenanceActionsGrid

This is the main action surface.

Core cards:

- Diagnostics Scan
- Contract Verification
- Drift Detection
- Coverage Audit
- Template Integrity Check
- Standards Snapshot Check

Each card should show:

- action name
- purpose
- status
- last run time
- result summary
- launch button

That makes the page feel operational instead of informational.

VerificationAndAuditRow

This section is for more detailed system confidence work.

Panels:

- verification results summary

- coverage audit summary
- contract mismatches
- unresolved drift items

This is where the page starts feeling serious and engineering-driven.

`RecoveryAndSnapshotsRow`

This section is for controlled recovery tools.

Panels:

- available snapshots
- last restore point
- rollback readiness
- cleanup actions

You do not need real destructive functionality in the first pass, but the structure should exist.

Suggested first-pass data model

```
type MaintenanceTask = {
 id: string;
 label: string;
 status: "success" | "warning" | "error" | "blocked" | "idle";
 summary: string;
 lastRun?: string;
};
```

Files to create

- `App/src/pages/system/maintenance/MaintenancePage.tsx`
- `App/src/pages/system/maintenance/components/MaintenanceActions.tsx`
- `App/src/pages/system/maintenance/components/DriftScannerCard.tsx`
- `App/src/pages/system/maintenance/components/ContractVerifierCard.tsx`
- `App/src/pages/system/maintenance/components/CoverageAuditCard.tsx`
- `App/src/pages/system/maintenance/components/SnapshotManagerCard.tsx`

Then update:

- `App/src/pages/maintenance.tsx`

to become a wrapper.

`maintenance.tsx` wrapper

```
import { MaintenancePage } from "@/pages/system/maintenance/MaintenancePage";

export default function LegacyMaintenancePage() {
 return <MaintenancePage />;
}
```

First-pass visual feel

Maintenance should feel:

- controlled
- deliberate

- high-trust
- restricted
- powerful without being flashy

This is the “engineering bay” of the system.

First-pass build order

Build in this order:

1. `MaintenancePage.tsx`
2. `MaintenanceActions.tsx`
3. `ContractVerifierCard.tsx`
4. `DriftScannerCard.tsx`
5. `CoverageAuditCard.tsx`
6. `SnapshotManagerCard.tsx`

That gets the control surface in place quickly.

What this completes

Once Maintenance is in, you have the main operational backbone of the new UI:

- Workbench
- Artifact Explorer
- Health

- Logs
- Maintenance

That is the point where the suite stops being just a nicer-looking dashboard and becomes a full working lab environment.

New Run / new-assembly.tsx overhaul

`New Run` / `new-assembly.tsx` overhaul

Purpose

Turn the current intake flow into a real **run configuration console**.

This page should feel like:

- setting up a mission
- configuring a system run
- validating readiness before launch

Not just filling out a long form.

This matters because this is the front door to the whole suite.
If the rest of the system feels premium and powerful, the intake flow has to match it.

Layout

Use a **3-zone configuration layout**.

```
NewRunPage
├── LaunchToolbar
├── NewRunWorkspace
│   ├── StepSidebar
│   ├── IntakeFormCanvas
│   └── LiveSummaryPanel
```

`LaunchToolbar`

Top strip with:

- page title

- draft status
- progress %
- validation summary
- launch action

Actions:

- Save Draft
- Reset
- Review
- Start Run

This makes the page feel operational immediately.

StepSidebar

This becomes the run setup navigator.

Use the steps you already have, just present them better.

Steps

- Project
- Category
- Intent
- Routing
- Design
- Functional

- Data
- Integrations
- Auth
- NFR
- Review

Each step should show

- title
- completion state
- warning state if incomplete
- active state

This should feel like a control checklist, not a plain wizard tracker.

`IntakeFormCanvas`

This is the main form area.

It should:

- keep the current step component structure
- wrap each step in a premium panel
- improve spacing and section hierarchy
- make inputs feel cleaner and more deliberate
- surface helper text better

You already have good step separation in the current intake components, so this is more about shell and presentation first than rewriting the form logic.

`LiveSummaryPanel`

This is the big upgrade.

This right panel should show:

- completion percentage
- missing required sections
- current inferred run shape
- selected category / intent / scope snapshot
- warnings
- launch readiness

This is what makes the page feel intelligent and useful.

Without it, the flow is just a multi-step form.

Suggested first-pass structure

```
NewRunPage
├── LaunchToolbar
├── grid
│   ├── StepSidebar
│   ├── active intake step
│   └── LiveSummaryPanel
```

On smaller screens, this can stack.

What to keep from current implementation

Keep:

- current route
- current intake wizard logic
- current step components
- existing state management
- existing submission behavior

Do **not** rewrite the whole intake system first.

This should be an upgrade pass, not a rebuild.

Files to create

- `App/src/pages/new-run/NewRunPage.tsx`
- `App/src/pages/new-run/components/StepSidebar.tsx`
- `App/src/pages/new-run/components/LiveSummaryPanel.tsx`
- `App/src/pages/new-run/components/ReviewLaunchPanel.tsx`
- `App/src/pages/new-run/components/LaunchToolbar.tsx`

Then update:

- `App/src/pages/new-assembly.tsx`

to become a wrapper.

`new-assembly.tsx` wrapper

```
import { NewRunPage } from "@/pages/new-run/NewRunPage";

export default function LegacyNewAssemblyPage() {
 return <NewRunPage />;
}
```

Suggested first-pass data surfaces

The summary panel should derive and display things like:

- current step
- completed step count
- missing critical steps
- selected project type
- selected category
- selected auth mode
- integration count
- risk / complexity hint

Even if some of that starts as simple computed text, it adds a lot of value.

First-pass visual feel

The New Run page should feel:

- guided
- premium
- clear
- structured

- launch-ready

Think:
configuration bay before execution

Not:

- a survey form
- a generic onboarding flow
- a flat admin wizard

Build order

Build in this order:

1. `NewRunPage.tsx`
2. `LaunchToolbar.tsx`
3. `StepSidebar.tsx`
4. `LiveSummaryPanel.tsx`
5. `ReviewLaunchPanel.tsx`

Then wire the existing intake wizard into the center panel.

What this completes

Once New Run is upgraded, your main workflow loop is fully covered:

- start a run

- operate the run in Workbench
- inspect outputs in Artifacts
- monitor system state in Health
- trace events in Logs
- maintain the platform in Maintenance

That is the core lab OS.

RunsPage

RunsPage

Purpose

Separate **fleet management** from the Command Center.

Right now Command Center is doing two jobs:

- overall mission control
- run management

That overlap will get messy.
`RunsPage` should become the place where you manage, inspect, filter, and compare all runs.

Command Center stays high-level.
Runs becomes operational.

Layout

Use a **3-zone management layout**.

```
RunsPage
├── RunsToolbar
├── RunsWorkspace
│   ├── RunsFilters
│   ├── RunsTable or RunsCardGrid
│   └── RunInspectorPanel
```

`RunsToolbar`

Top strip with:

- page title

- search
- saved view placeholder
- quick actions

Actions:

- New Run
- Resume Latest
- Compare Runs
- Export Selected

This makes the page feel like a working surface, not a list dump.

RunsFilters

Left column for narrowing the run set.

First-pass filters

- status
- current stage
- gate state
- date range
- environment
- run type/category

Keep them visually strong even if the first pass has simple filtering logic.

`RunsTable` or `RunsCardGrid`

This is the main run list.

First pass should use a **table**, because that is better for dense operator workflows.

Columns

- Run Name
- Run ID
- Status
- Current Stage
- Gate State
- Started
- Duration
- Artifacts
- Actions

Each row should feel actionable.

Row actions

- Open Workbench
- Resume
- Export
- Compare
- Delete later if needed

`RunInspectorPanel`

Right-side contextual panel for the selected run.

Show:

- run name
- run ID
- status
- current stage
- duration
- gate summary
- artifact count
- started time
- latest event
- quick action buttons

This gives the page depth and makes it feel like part of the same workstation system as Workbench.

Suggested first-pass data model

```
type RunListItem = {
 id: string;
 name: string;
 status: "idle" | "running" | "success" | "warning" | "error" | "blocked";
 stage: string;
 gateState?: string;
 startedAt?: string;
 duration?: string;
 artifacts?: number;
 environment?: string;
```

};

Files to create

- `App/src/pages/runs/RunsPage.tsx`
- `App/src/pages/runs/components/RunsToolbar.tsx`
- `App/src/pages/runs/components/RunsFilters.tsx`
- `App/src/pages/runs/components/RunsTable.tsx`
- `App/src/pages/runs/components/RunInspectorPanel.tsx`

Route plan

You can add:

/runs

Then update sidebar nav so:

- `Command Center` → `/`
- `Runs` → `/runs`

That removes the current overlap.

First-pass behavior

- click a row
- row becomes selected

- inspector updates
- open Workbench from row or inspector
- filters narrow visible runs

That is enough for the first real version.

Visual feel

Runs should feel:

- dense
- premium
- fast
- controllable
- operational

Think:
fleet management console for all executions

Not:

- plain admin table
- reporting dashboard
- spreadsheet clone

Build order

1. `RunsPage.tsx`

2. `RunsTable.tsx`
3. `RunInspectorPanel.tsx`
4. `RunsFilters.tsx`
5. `RunsToolbar.tsx`

That gets the main value in fastest.

Command Center second pass

Command Center second pass

Purpose

Now that `RunsPage` exists, Command Center can stop trying to be both:

- mission control
- run manager

This second pass should make Command Center a true **executive lab dashboard** focused on:

- situational awareness
- active alerts
- quick launch actions
- system-wide summaries
- “what needs attention right now”

Not dense fleet operations. That belongs to Runs.

What changes in this pass

Command Center should shift from:

- list-heavy
- semi-operational
- overlapping with Runs

to:

- high-level
- live
- focused
- action-oriented

It becomes the page you open first, then branch into:

- Runs
- Workbench
- Health
- Logs
- Maintenance

Layout

Use a **mission-control dashboard layout**:

```
CommandCenterPage
├── HeroStatusBar
├── QuickActionsRow
├── MetricsGrid
├── ActiveAttentionRow
│   ├── ActiveRunsPanel
│   ├── AlertsPanel
│   └── SystemHealthPanel
├── ActivityAndOutputRow
│   ├── RecentActivityPanel
│   └── RecentOutputsPanel
```

---

1. `HeroStatusBar`

This is the top identity band.

Show

- AXION Command Center
- overall system state
- active run count
- gate warning count
- degraded subsystem count
- last major event time

This should feel like the “all systems overview.”

---

2. `QuickActionsRow`

This should be highly visible.

Actions

- Start New Run
- Open Runs
- Resume Latest Run
- Review Gate Failures
- Open Logs
- Run Diagnostics

These should be fast-launch controls, not buried buttons.

---

3. `MetricsGrid`

These are your executive stats.

Suggested cards

- Active Runs
- Completed Today
- Failed or Blocked Runs
- Artifacts Ready
- Gate Warnings
- System Health Score or Status

These should stay concise and high-value.

---

4. `ActiveRunsPanel`

This is not the full Runs table anymore.

It should show:

- only a few active or recently changed runs
- name
- status
- current stage
- quick open action

Think of it as:
priority run snapshot, not run management.

---

5. `AlertsPanel`

This should surface only the most important issues.

Show things like

- blocked gate
- failed verification
- degraded subsystem
- export issue
- maintenance warning

Each alert should include:

- severity
- short summary
- target action

This page should help you decide where to go next.

---

6. `SystemHealthPanel`

Compact system status snapshot.

Show

- healthy / degraded / failing
- top subsystem statuses

- quick link to Health
- quick link to Maintenance

This keeps Command Center connected to the rest of the system.

---

7. `RecentActivityPanel`

Live or recent timeline preview.

Show

- run started
- stage completed
- gate failed
- artifact exported
- maintenance scan completed

Only a handful of high-value events should appear here.

---

8. `RecentOutputsPanel`

Show recently generated useful outputs:

- packaged kits
- canonical specs
- gate reports
- exports

- proof artifacts

This gives the dashboard a real production feel.

Suggested files to create or expand

If you want to split it cleanly:

- `App/src/pages/command-center/CommandCenterPage.tsx`
- `App/src/pages/command-center/sections/HeroStatusBar.tsx`
- `App/src/pages/command-center/sections/MetricsGrid.tsx`
- `App/src/pages/command-center/sections/ActiveRunsPanel.tsx`
- `App/src/pages/command-center/sections/AlertsPanel.tsx`
- `App/src/pages/command-center/sections/SystemHealthPanel.tsx`
- `App/src/pages/command-center/sections/RecentActivityPanel.tsx`
- `App/src/pages/command-center/sections/RecentOutputsPanel.tsx`
- `App/src/pages/command-center/sections/QuickActionsPanel.tsx`

What to remove from Command Center now

Once Runs exists, Command Center should **not** try to become:

- a full run table
- a heavy filtering surface

- a dense inspection workspace

Those belong elsewhere.

Command Center should always answer:

- what is happening
- what changed
- what is wrong
- what should I do next

Visual feel

Command Center should feel:

- high-level
- premium
- live
- confident
- uncluttered

Think:
Tony Stark overview wall
not:
operator detail grid

The dense engineering work happens deeper in the product.

Build order

1. `HeroStatusBar`
2. `QuickActionsPanel`
3. `MetricsGrid`
4. `AlertsPanel`
5. `ActiveRunsPanel`
6. `SystemHealthPanel`
7. `RecentActivityPanel`
8. `RecentOutputsPanel`

That gets the identity right first.

What this gives you

Once this second pass is done, your product layers become clean:

- **Command Center** = overview and direction
- **Runs** = fleet management
- **Workbench** = single-run operations
- **Artifacts** = outputs
- **Health** = diagnostics
- **Logs** = observability
- **Maintenance** = repair and verification
- **New Run** = launch configuration

That is a much stronger product shape.

Knowledge / Library consolidation pass

Knowledge / Library consolidation pass

Purpose

Right now the knowledge side of the suite is useful, but it is fragmented.

You have many separate library pages:

- system
- orchestration
- gates
- policy
- intake
- canonical
- standards
- templates
- planning
- verification
- kit
- telemetry
- audit

That works for raw access, but it does **not** feel like one intelligence surface.

This pass turns that cluster into a real **Knowledge System**.

Goal

Move from:

- many isolated doc pages
- flat utility navigation
- internal-tool feeling

to:

- one knowledge home
- category-driven navigation
- better browsing and discovery
- cleaner doc reading experience
- stronger “system intelligence” feel

New structure

Use a 3-layer knowledge model:

```
Knowledge
├── KnowledgeHomePage
├── KnowledgeCategoryPage
└── KnowledgeDocViewerPage
```

Layer 1 — Knowledge Home

The entry point.

Layer 2 — Knowledge Category

A filtered category view like Templates, Gates, Standards, Policy.

Layer 3 — Knowledge Doc Viewer

The page where a specific document is read and inspected.

---

1. `KnowledgeHomePage`

Purpose

Central landing page for all system knowledge.

Should show

- search
- category grid
- recently viewed docs
- pinned docs
- changed docs
- coverage/health summary
- quick links to high-value sections

This page should feel like the system's **intelligence library**, not a stack of links.

---

2. `KnowledgeCategoryPage`

Purpose

Make each library area feel organized and browsable.

Example categories

- System
- Orchestration
- Gates
- Policy
- Intake
- Canonical
- Standards
- Templates
- Planning
- Verification
- Kit
- Telemetry
- Audit

Each category page should show

- category title
- short description
- document list
- status/health indicators
- recently changed docs
- quick filters later if needed

This gives the content structure without forcing a total rewrite of the doc content itself.

---

3. `KnowledgeDocViewerPage`

Purpose

Make reading a specific library doc feel intentional and premium.

Layout

Use a doc workstation layout:

```
KnowledgeDocViewerPage
├── DocToolbar
└── grid
  ├── KnowledgeSidebar
  ├── KnowledgeDocViewer
  └── KnowledgeMetaPanel
```

Left

Knowledge sidebar

- categories
- docs in current category
- current selection

Center

The document viewer

- rich content
- headings
- clean reading layout

Right

Metadata / context

- category
- last updated
- related docs
- doc health
- quick navigation links

---

4. Migration strategy

Do **not** rewrite all library content at once.

Use the same safe migration pattern you used elsewhere.

Pattern

1. create new knowledge shell pages
2. keep old library pages alive
3. render old content inside the new category/doc viewer structure
4. consolidate gradually
5. only later retire old standalone pages

That keeps this pass from becoming a giant detour.

---

5. Route target

Add a cleaner knowledge route structure:

/knowledge
/knowledge/:category

/knowledge/:category/:doc

Examples:

- `/knowledge`
- `/knowledge/templates`
- `/knowledge/gates`
- `/knowledge/policy`

You can keep old routes working temporarily and map them into the new system later.

---

6. First-pass file structure

Create:

- `App/src/pages/intelligence/knowledge/KnowledgeHomePage.tsx`
- `App/src/pages/intelligence/knowledge/KnowledgeCategoryPage.tsx`
- `App/src/pages/intelligence/knowledge/KnowledgeDocViewerPage.tsx`

And components:

- `App/src/pages/intelligence/knowledge/components/KnowledgeSearch.tsx`
- `App/src/pages/intelligence/knowledge/components/KnowledgeCategoryGrid.tsx`
- `App/src/pages/intelligence/knowledge/components/KnowledgeDocList.tsx`

- `App/src/pages/intelligence/knowledge/components/KnowledgeDocViewer.tsx`
- `App/src/pages/intelligence/knowledge/components/KnowledgeSidebar.tsx`
- `App/src/pages/intelligence/knowledge/components/KnowledgeMetaPanel.tsx`

---

7. Category model

Use a simple first-pass type:

```
type KnowledgeCategory = {
 id:
   | "system"
   | "orchestration"
   | "gates"
   | "policy"
   | "intake"
   | "canonical"
   | "standards"
   | "templates"
   | "planning"
   | "verification"
   | "kit"
   | "telemetry"
   | "audit";
 label: string;
 description: string;
 docCount?: number;
 status?: "healthy" | "warning" | "stale";
};
```

---

8. Doc model

```
type KnowledgeDoc = {
 id: string;
 title: string;
```

```
 category: string;
 summary?: string;
 updatedAt?: string;
 status?: "current" | "warning" | "stale";
};
```

This is enough to build the UI before deeper content wiring.

---

9. What to do with the current library pages

Current pages

- `system-library.tsx`
- `orchestration-library.tsx`
- `gates-library.tsx`
- `policy-library.tsx`
- `intake-library.tsx`
- `canonical-library.tsx`
- `standards-library.tsx`
- `templates-library.tsx`
- `planning-library.tsx`
- `verification-library.tsx`
- `kit-library.tsx`
- `telemetry-library.tsx`
- `audit-library.tsx`

Recommended handling

Do not delete them yet.

Instead, treat each one as a category content source until the new knowledge pages are stable.

---

10. First-pass behavior

Knowledge Home

- click a category card
- open category page

Knowledge Category

- see list of docs in that category
- click a doc
- open viewer

Doc Viewer

- show doc content
- show metadata
- show related docs later

That is enough for a strong first version.

---

11. Visual feel

Knowledge should feel:

- curated
- structured
- premium
- reference-oriented
- calm and readable

This is not a heavy ops page.
It should feel more like a **system intelligence archive**.

Use:

- strong reading layout
- softer panels
- clearer typography
- less density than Logs or Runs

---

12. Build order

Build in this order:

1. `KnowledgeHomePage.tsx`
2. `KnowledgeCategoryGrid.tsx`
3. `KnowledgeCategoryPage.tsx`
4. `KnowledgeDocList.tsx`
5. `KnowledgeDocViewerPage.tsx`
6. `KnowledgeSidebar.tsx`

7. `KnowledgeDocViewer.tsx`

8. `KnowledgeMetaPanel.tsx`

That gets structure first, then reading experience.

---

13. Sidebar/navigation update after this

Once Knowledge exists, update the main sidebar from:

- `Libraries`

to:

- `Knowledge`

And point it to:

- `/knowledge`

That will make the product feel much cleaner.

---

14. What this completes

Once Knowledge is consolidated, the product shape becomes complete:

- Command Center
- Runs
- Workbench
- New Run

- Artifacts
- Health
- Logs
- Maintenance
- Knowledge
- Export

At that point, the big UI architecture is in place.

Export page second pass

Export page second pass

Purpose

Turn Export into a real **packaging dock** instead of a simple output page.

By this point, the system can:

- configure runs
- operate runs
- inspect artifacts
- monitor health
- trace logs
- manage maintenance
- browse knowledge

So Export now needs to become the place where you:

- package deliverables
- choose export scope
- preview what will be included
- download ready bundles
- review export history

This is the final output surface of the lab.

Goal

Move Export from:

- basic utility page

to:

- controlled release surface
- package staging area
- output handoff station

Layout

Use a **3-zone dock layout**:

```
ExportPage
├── ExportToolbar
├── ExportStatusBand
└── ExportWorkspace
  ├── ExportOptions
  ├── PackageManifestPreview
  └── ExportQueue / ExportHistory
```

---

1. `ExportToolbar`

Top strip with:

- page title
- selected run or scope
- export target summary
- quick actions

Actions

- Export Full Kit
- Export Docs Only
- Export Proof Only
- Export Selected Artifacts
- Refresh Queue

This should feel like release control.

---

2. `ExportStatusBand`

Hero strip showing:

- ready packages
- queued exports
- failed exports
- last completed export
- current selected package type

This should quickly answer:

- what is ready
- what is pending
- what failed

---

3. `ExportOptions`

This is the left-side configuration panel.

Show

- export type selector
- include/exclude toggles
- current run selector
- output format summary
- package notes / label later

First-pass export types

- Full Kit
- Docs Only
- Proof Only
- Artifacts Only
- Snapshot

Even if actions are mocked at first, the UI should feel deliberate.

---

4. `PackageManifestPreview`

This is the center panel.

Purpose

Let you preview what will be exported before launching it.

Show

- package name

- included sections
- file count
- artifact count
- generated manifest preview
- run/source references

This is what makes Export feel serious instead of blind.

---

5. `ExportQueue`

This shows active and pending exports.

Each item should show

- export name
- type
- status
- created time
- source run
- completion state

Statuses:

- queued
- processing
- ready
- failed

This makes the page feel alive.

---

6. `ExportHistory`

Below or beside queue, show recent completed exports.

Each row/card should show

- export name
- type
- completed time
- source run
- download action
- retry later if needed

This gives the page continuity.

Suggested first-pass data model

```
type ExportJob = {
 id: string;
 label: string;
 type: "full-kit" | "docs-only" | "proof-only" | "artifacts-only" | "snapshot";
 status: "queued" | "processing" | "ready" | "failed";
 runId?: string;
 createdAt?: string;
 completedAt?: string;
};
```

Files to create or expand

- `App/src/pages/export/ExportPage.tsx`
- `App/src/pages/export/components/ExportToolbar.tsx`
- `App/src/pages/export/components/ExportOptions.tsx`
- `App/src/pages/export/components/PackageManifestPreview.tsx`
- `App/src/pages/export/components/ExportQueue.tsx`
- `App/src/pages/export/components/ExportHistory.tsx`

Then update:

- `App/src/pages/export.tsx`

to point cleanly at the upgraded page if it does not already.

First-pass behavior

- choose export type
- select package scope
- manifest preview updates
- queued export appears in queue/history surface
- ready item shows download action placeholder

That is enough for first pass.

Visual feel

Export should feel:

- controlled
- polished
- release-oriented
- final-step professional

Think:
shipping dock for generated outputs

Not:

- random download page
- raw file list
- generic utility panel

Build order

1. `ExportPage.tsx`
2. `ExportOptions.tsx`
3. `PackageManifestPreview.tsx`
4. `ExportQueue.tsx`
5. `ExportHistory.tsx`
6. `ExportToolbar.tsx`

That gets the workflow in place quickly.

What this completes

Once Export gets its second pass, the full UI overhaul is structurally covered:

- Command Center
- Runs
- Workbench
- New Run
- Artifacts
- Health
- Logs
- Maintenance
- Knowledge
- Export

That is the full Lab OS surface.

Polish layer

Polish layer

Purpose

This is the pass that makes the system feel finished.

By now the core surfaces are defined:

- Command Center
- Runs
- Workbench
- New Run
- Artifacts
- Health
- Logs
- Maintenance
- Knowledge
- Export

The polish layer is what turns that from a strong internal app into a **premium lab operating environment**.

This pass is about:

- speed
- cohesion
- responsiveness
- comfort

- delight without fluff

Main polish modules

```
Polish Layer
├── Command Palette
├── Global Search
├── Notification Center
├── Motion System
├── Density Toggle
├── Saved Views
└── Keyboard Shortcuts
```

---

1. Command Palette

Purpose

Give you a fast universal launcher.

This is one of the biggest “Tony Stark” feeling upgrades because it makes the app feel like a real operating system.

Trigger

- `Ctrl + K`

Actions to include first

- Start New Run
- Open Runs
- Open Workbench
- Open Artifacts

- Open Health
- Open Logs
- Open Maintenance
- Open Knowledge
- Open Export
- Run Diagnostics
- Review Gate Failures

Files to create

- `App/src/components/command/CommandPalette.tsx`
- `App/src/hooks/use-command-palette.ts`

---

2. Global Search

Purpose

Search across the whole suite.

Search targets

- runs
- artifacts
- docs
- logs

- templates later
- gates later

First-pass behavior

- type query
- grouped results
- click result to navigate

Files to create

- `App/src/components/command/GlobalSearch.tsx`
- `App/src/hooks/use-global-search.ts`

---

3. Notification Center

Purpose

Surface important system changes without forcing you to hunt for them.

Should show

- gate failures
- run completed
- export ready
- degraded subsystem

- maintenance results
- warnings

Behavior

- dropdown tray from topbar
- unread count
- severity styling
- click through to destination

Files to create

- `App/src/components/feedback/NotificationCenter.tsx`

---

4. Motion system

Purpose

Make the UI feel alive and premium without turning into a toy.

Use motion for

- route transitions
- inspector updates
- drawer openings
- stage selection

- hover response
- status changes

Rules

- short and restrained
- informative, not decorative
- no loud sci-fi gimmicks

Files to add later

- `App/src/styles/motion.css`
- or motion wrappers where needed

---

5. Density toggle

Purpose

Let the workspace adapt to how you want to work.

Modes

- **Focus mode** = larger spacing, calmer panels
- **Operator mode** = denser rows, more visible data

This is especially useful for:

- Runs

- Logs
- Workbench
- Knowledge

Files to create

- `App/src/components/ui/density-toggle.tsx`
- `App/src/hooks/use-density-mode.ts`

---

6. Saved views

Purpose

Let you save common operational filters.

Best first places to support it

- Runs
- Logs
- Artifacts
- Knowledge later

Example saved views

- Active Runs
- Failed Gates

- Recent Exports
- Last 24h Errors

This makes daily workflows much faster.

---

7. Keyboard shortcuts

Purpose

Make the suite feel like a real workstation.

High-value shortcuts

- `Ctrl + K` → command palette
- `/` → focus search
- `g r` → Runs
- `g h` → Health
- `g l` → Logs
- `g a` → Artifacts
- `g w` → latest Workbench
- `[` and `]` → previous/next run or section
- `Esc` → close drawers/palette

You do not need every shortcut at once. Start with the highest-value ones.

Best order for the polish pass

First

- Command Palette
- Notification Center

Then

- Global Search
- Keyboard shortcuts

Then

- Density Toggle
- Saved Views

Last

- Motion refinement across the app

That order gives the biggest perceived value first.

What this completes

Once the polish layer is in, the app stops feeling like:

- a well-designed internal dashboard

and starts feeling like:

- a complete premium operating system for your lab

Final full UI overhaul stack

1. Shell
2. Workbench
3. Pipeline + Gates + Inspector
4. Artifacts
5. Health
6. Logs
7. Maintenance
8. New Run
9. Runs
10. Command Center second pass
11. Knowledge
12. Export
13. Polish Layer

That is the full overhaul path.

Master implementation roadmap

Master implementation roadmap

This is the full build order for the UI overhaul, organized into clean execution phases so the work stays controlled.

Phase 1 — Foundation shell

Goal

Replace the bland dev shell with the new Lab OS frame.

Deliverables

- `AppShell`
- `Sidebar`
- `Topbar`
- `RightInspector`
- shared UI primitives:
  - `glass-panel`
  - `status-chip`
  - `metric-card`
  - `toolbar`

Completion checkpoint

- app renders inside new shell
- current routes still work
- visual identity shift is obvious
- no major route breakage

Phase 2 — Workbench core

Goal

Turn `assembly.tsx` into the new main operating surface.

Deliverables

- `WorkbenchPage`
- `WorkbenchHeader`
- `WorkbenchTabRail`
- `StageRail`
- `WorkbenchOverview`
- `assembly.tsx` becomes wrapper

Completion checkpoint

- `/assembly/:id` opens new Workbench shell
- stage rail is visible

- overview tab works
- page already feels like a workstation

Phase 3 — Workbench depth

Goal

Make the Workbench operational, not just styled.

Deliverables

- `WorkbenchPipeline`
- `StageDetailCard`
- `WorkbenchGates`
- stage/gate selection model
- Workbench-local inspector behavior
- upgraded `StageRail`

Completion checkpoint

- stage click updates detail context
- gate click updates detail context
- pipeline tab is usable
- gates tab is usable
- Workbench starts feeling real

Phase 4 — Command Center first pass

Goal

Upgrade the landing page into a real mission-control surface.

Deliverables

- `CommandCenterPage`
- `dashboard.tsx` wrapper
- hero band
- metrics grid
- active runs snapshot
- alerts panel
- recent activity panel

Completion checkpoint

- `/` feels premium
- app opens into a clear high-level overview
- dashboard no longer feels like a plain admin panel

Phase 5 — Artifacts

Goal

Turn `files.tsx` into the artifact lab.

Deliverables

- `ArtifactExplorerPage`
- `ArtifactTree`
- `ArtifactPreview`
- `ArtifactInspector`
- `files.tsx` wrapper

Completion checkpoint

- browse artifact tree
- select file
- preview content
- inspect metadata
- artifact workflow feels connected to runs

Phase 6 — System state surfaces

Goal

Upgrade system monitoring and observability.

Deliverables

Health

- `HealthPage`
- system summary
- diagnostics grid
- trends
- action panel

Logs

- `LogsPage`
- filters
- log stream
- log inspector

Maintenance

- `MaintenancePage`
- diagnostics actions
- drift
- contract verification
- coverage audit
- snapshots

Completion checkpoint

- Health, Logs, and Maintenance all match the new visual system
- they feel actionable, not passive
- main operator surfaces are covered

Phase 7 — New Run

Goal

Make intake and launch feel like configuring a mission.

Deliverables

- `NewRunPage`
- `LaunchToolbar`
- `StepSidebar`
- `LiveSummaryPanel`
- `ReviewLaunchPanel`
- `new-assembly.tsx` wrapper

Completion checkpoint

- intake still works
- page feels premium
- summary panel adds real value

- launch flow matches rest of system

Phase 8 — Runs page

Goal

Separate fleet management from Command Center.

Deliverables

- `RunsPage`
- `RunsToolbar`
- `RunsFilters`
- `RunsTable`
- `RunInspectorPanel`
- `/runs` route
- sidebar update

Completion checkpoint

- runs are manageable in one dedicated place
- Command Center no longer has to do run management
- Workbench, Runs, and Command Center have clear roles

Phase 9 — Command Center second pass

Goal

Refocus Command Center into pure overview and direction.

Deliverables

- `HeroStatusBar`
- `QuickActionsPanel`
- refined `MetricsGrid`
- `SystemHealthPanel`
- `RecentOutputsPanel`
- smaller `ActiveRunsPanel`

Completion checkpoint

- Command Center is high-level only
- it tells you what matters now
- it points you into the correct deeper pages

Phase 10 — Knowledge consolidation

Goal

Turn scattered library pages into one intelligence system.

Deliverables

- `KnowledgeHomePage`
- `KnowledgeCategoryPage`
- `KnowledgeDocViewerPage`
- category grid
- doc list
- doc viewer
- metadata panel
- sidebar nav updated to `Knowledge`

Completion checkpoint

- knowledge browsing feels unified
- old scattered library feel is gone
- content is easier to navigate and read

Phase 11 — Export second pass

Goal

Turn Export into a real packaging dock.

Deliverables

- upgraded `ExportPage`
- `ExportOptions`
- `PackageManifestPreview`
- `ExportQueue`
- `ExportHistory`

Completion checkpoint

- export feels deliberate and controlled
- package preview exists
- queue/history make the page feel alive

Phase 12 — Polish layer

Goal

Make the app feel finished and fast.

Deliverables

- command palette
- global search
- notification center
- density toggle

- saved views
- keyboard shortcuts
- motion refinement

Completion checkpoint

- app feels premium and fluid
- navigation is fast
- frequent workflows are accelerated
- system feels like a real operating environment

Priority order

Tier 1

- shell
- Workbench core
- Workbench depth

Tier 2

- Command Center first pass
- Artifacts
- Health
- Logs

- Maintenance

Tier 3

- New Run
- Runs
- Command Center second pass

Tier 4

- Knowledge
- Export
- polish layer

Definition of “UI overhaul complete”

The overhaul is complete when:

- every major route lives inside the new shell
- Workbench is the true core operating page
- artifacts are previewable and inspectable
- health/logs/maintenance are premium and actionable
- New Run matches the rest of the product
- Runs is separated from Command Center
- Knowledge is unified

- Export feels like a release dock
- command palette/search/notifications exist
- the suite feels cohesive from top to bottom

Final product map

```
Command Center  -> overview
Runs            -> fleet management
Workbench       -> single-run operations
New Run         -> launch configuration
Artifacts       -> output inspection
Health          -> diagnostics
Logs            -> observability
Maintenance     -> verification / repair
Knowledge       -> system intelligence
Export          -> packaging / handoff
```